

Convolutional Neural Network

01

Convolutional Layer

CNN (Convolutional Neural Network)

- **합성곱** 신경망은 주로 이미지 인식과 같은 **컴퓨터 비전 분야**의 데이터를 분석하기 위해 사용되는 인공 신경망의 한 종류
- 입력 데이터의 **지역적인 특징**을 추출하는데 특화된 구조를 갖고 있으며, 이를 위해 **“합성곱(Convolution)” 연산**을 사용
- 합성곱 신경망(CNN)은 시각 분야에 국한되지 않고 음성인식이나 자연어처리 같은 다른 작업에도 많이 사용되는 대표적인 신경망

Convolution 연산

Convolution Filter를 이용한 연산

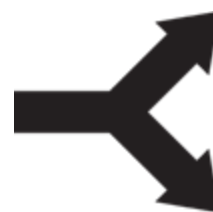
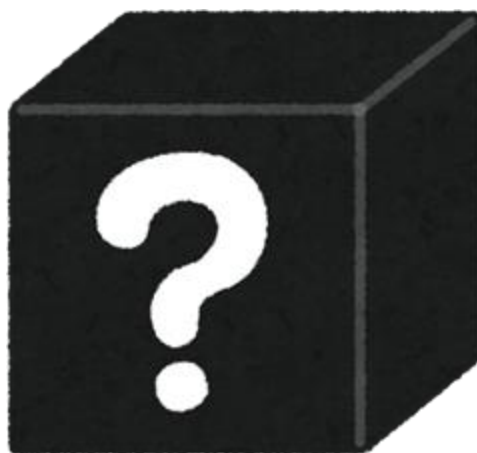
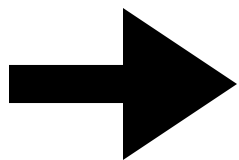


Convolutional Layer



Convolutional Neural Network

이미지 분류 모델

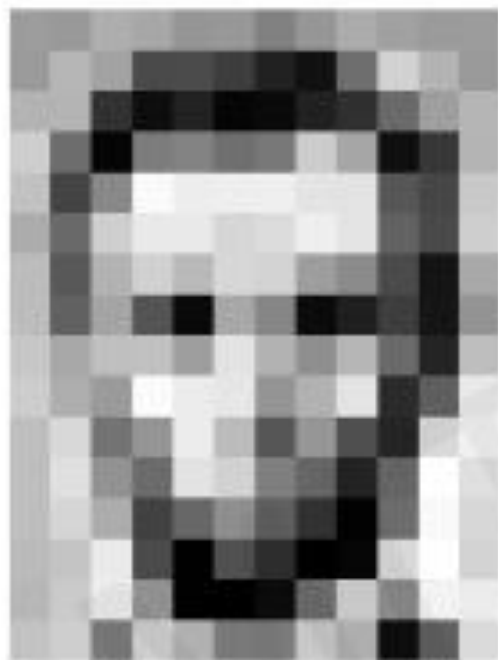


고양이

강아지

컴퓨터는 이미지를 어떻게 볼까?

Images are Numbers

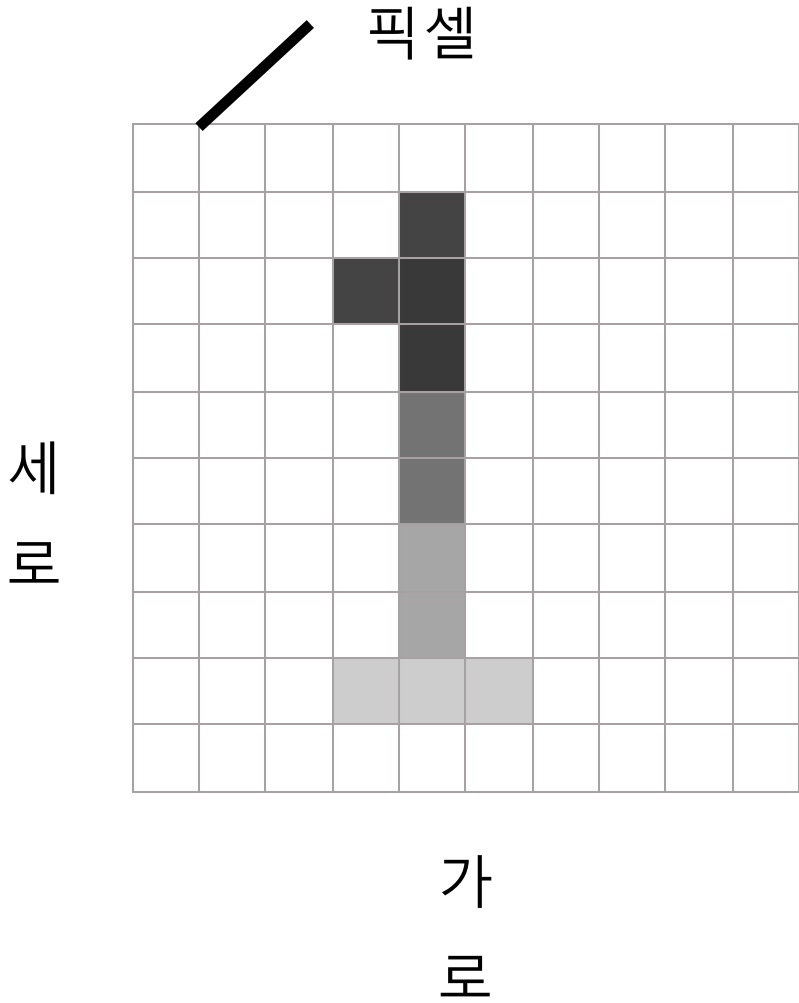


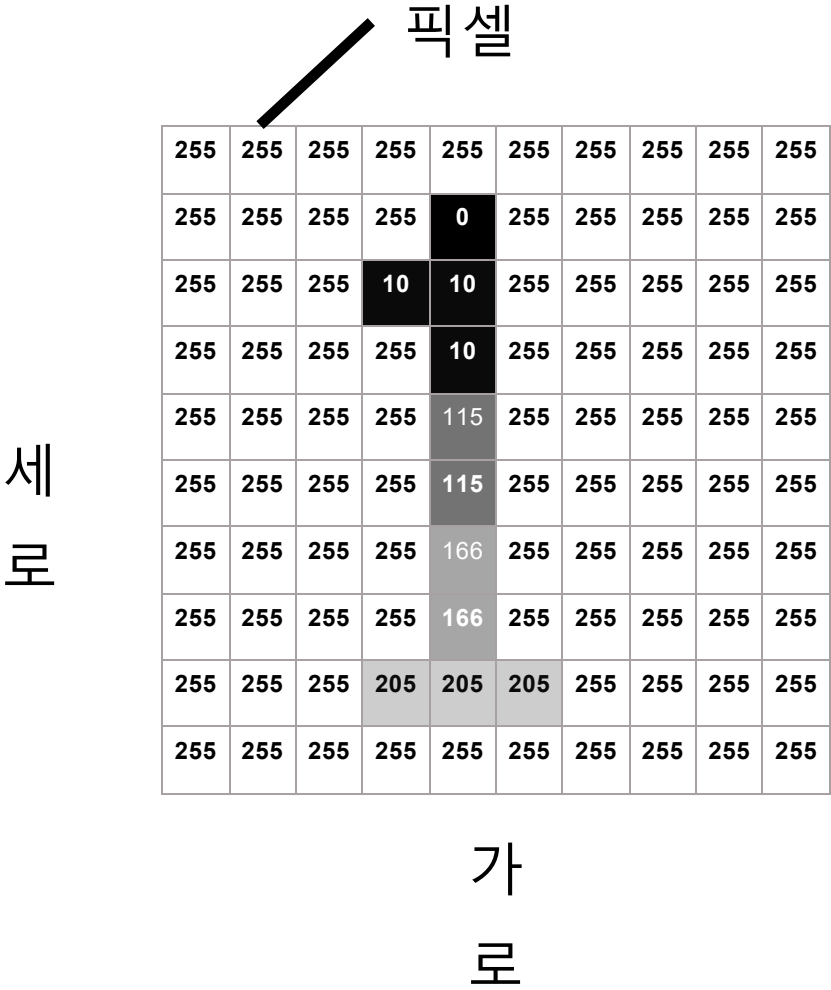
187	185	174	168	165	163	166	165	173	171	165	166
165	162	163	34	25	63	36	17	178	230	181	164
186	180	82	34	39	6	95	39	48	106	188	181
206	100	8	124	111	111	186	164	163	15	56	180
194	60	137	251	220	229	229	228	227	67	71	281
172	186	207	333	339	214	230	230	228	68	74	284
188	66	179	209	185	218	211	158	188	76	20	188
185	97	156	84	12	168	134	11	21	52	22	188
188	188	181	182	188	227	158	182	183	188	26	180
205	178	155	251	185	201	188	179	238	65	21	284
186	215	118	189	235	187	86	188	78	28	218	241
186	218	187	188	227	218	127	122	26	181	188	204
186	218	178	86	118	183	86	80	8	100	188	218
187	185	228	11	1	81	87	3	6	237	138	211
182	202	227	188	0	8	12	188	288	138	182	228
188	200	118	201	177	121	188	200	178	11	86	218

What the computer sees

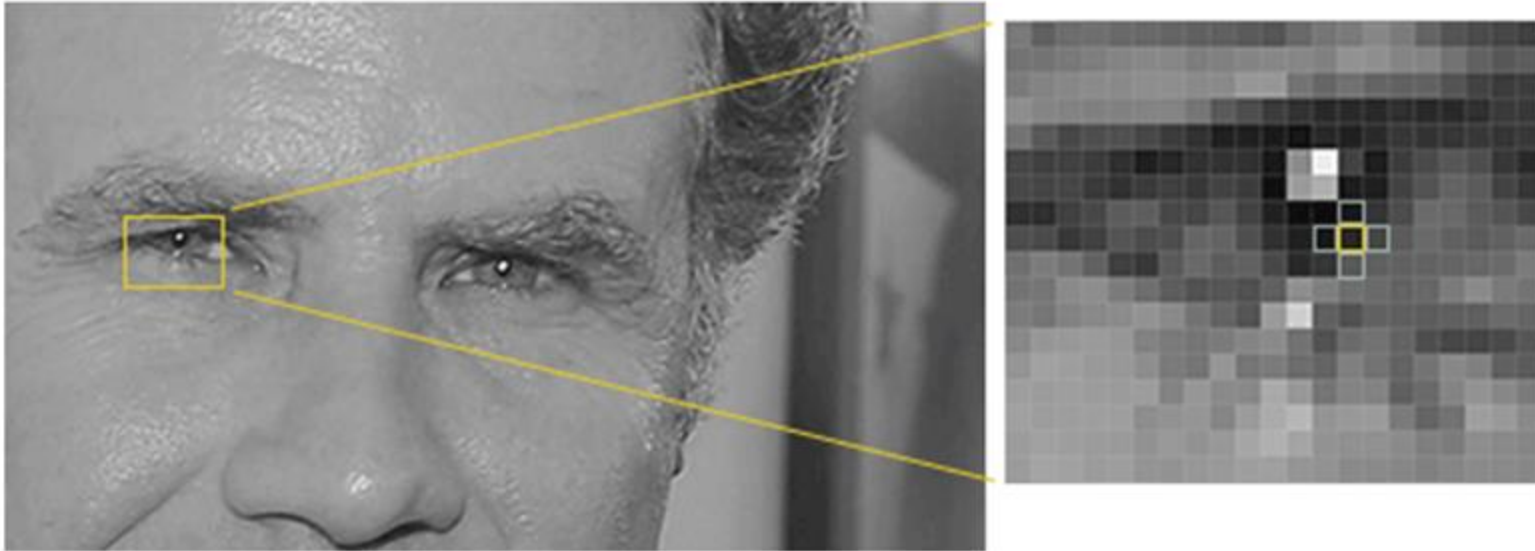
187	185	174	168	165	163	166	165	173	171	165	166
165	162	163	34	25	63	36	17	178	230	181	164
186	180	82	34	39	6	95	39	48	106	188	181
206	100	8	124	111	111	186	164	163	15	56	180
194	60	137	251	220	229	229	228	227	67	71	281
172	186	207	333	339	214	230	230	228	68	74	284
188	66	179	209	185	218	211	158	188	76	20	188
185	97	156	84	12	168	134	11	21	52	22	188
188	188	181	182	188	227	158	182	183	188	26	180
205	178	155	251	185	201	188	179	238	65	21	284
186	215	118	189	235	187	86	188	78	28	218	241
186	218	187	188	227	218	127	122	26	181	188	204
186	218	178	86	118	183	86	80	8	100	188	218
187	185	228	11	1	81	87	3	6	237	138	211
182	202	227	188	0	8	12	188	288	138	182	228
188	200	118	201	177	121	188	200	178	11	86	218

An image is just a matrix of numbers [0,255]!
i.e., 1080x1080x3 for an RGB image



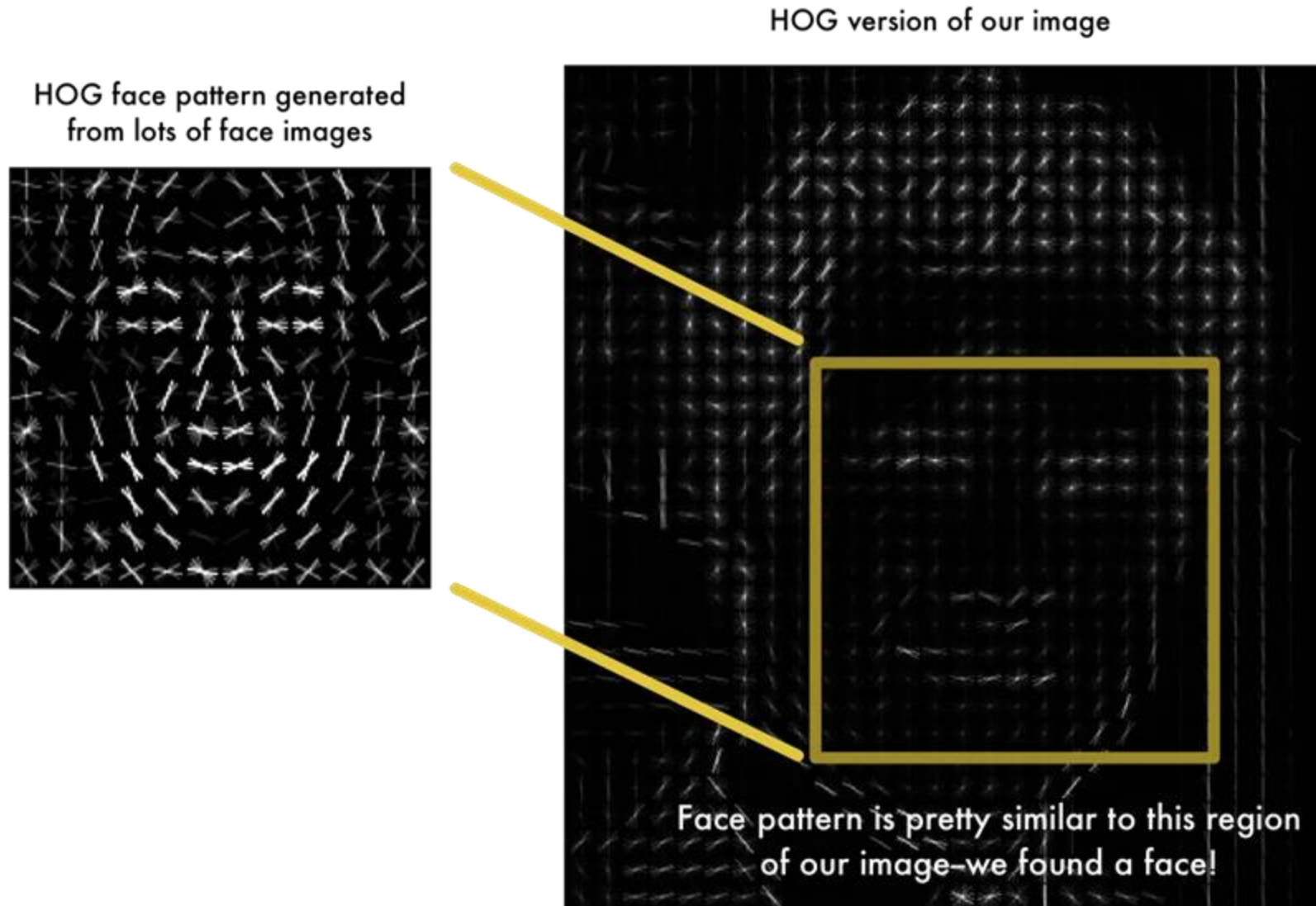


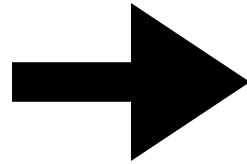
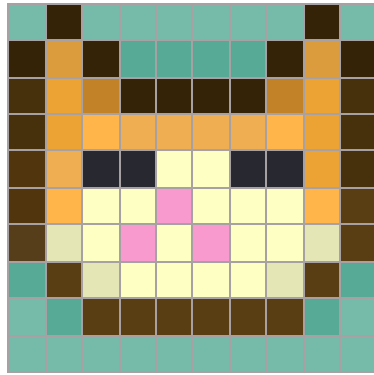
이미지 데이터

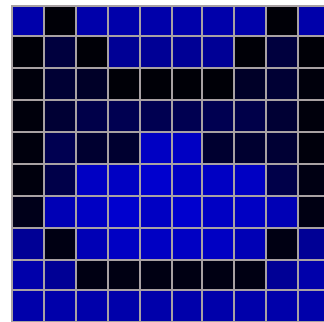
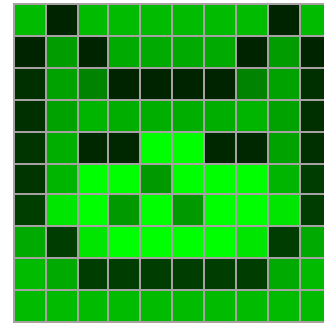
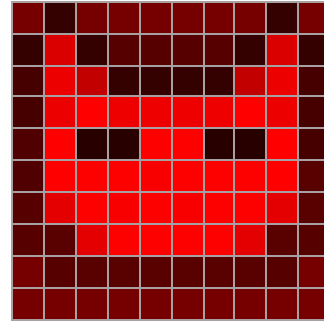
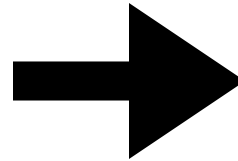


이미지 데이터

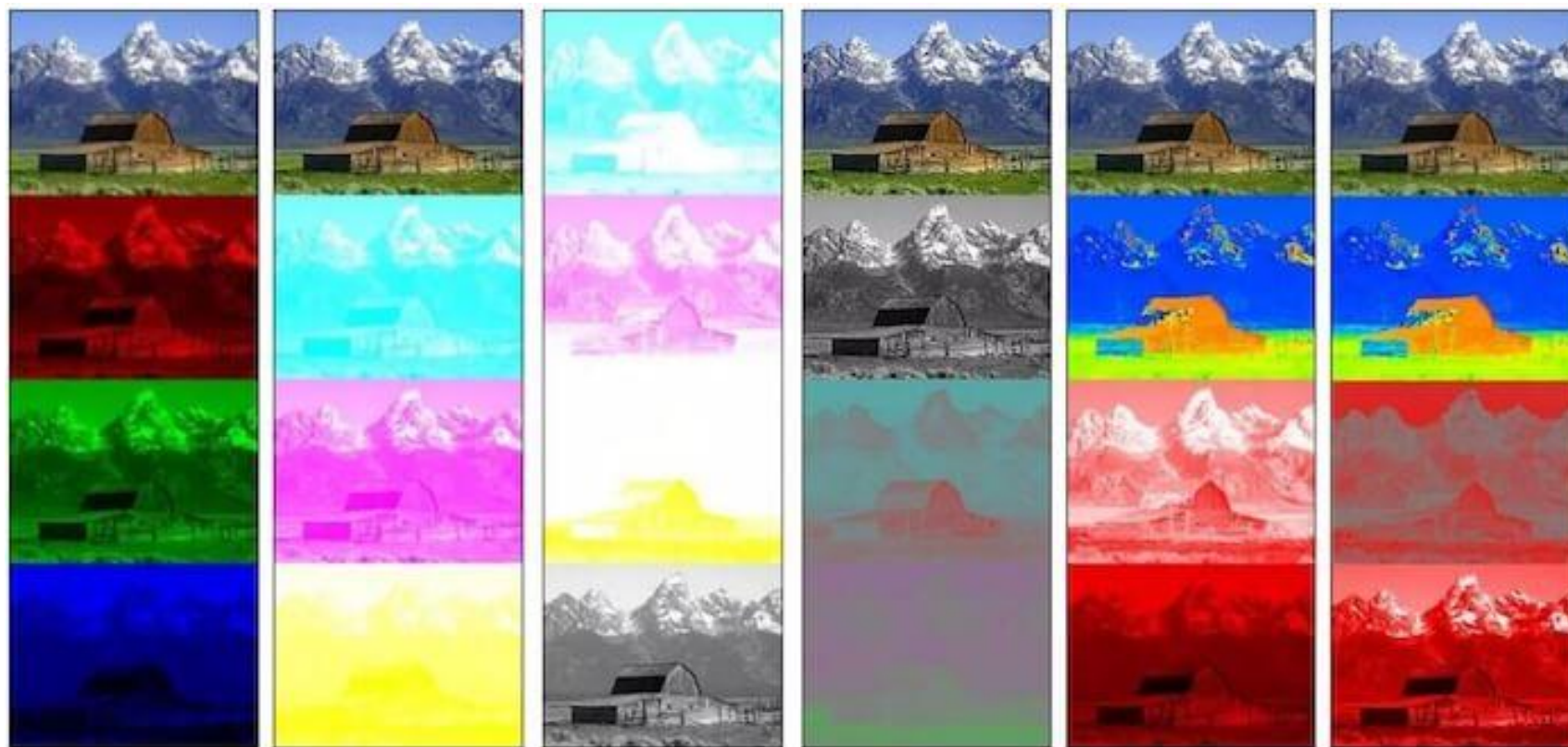








이미지 데이터



RGB

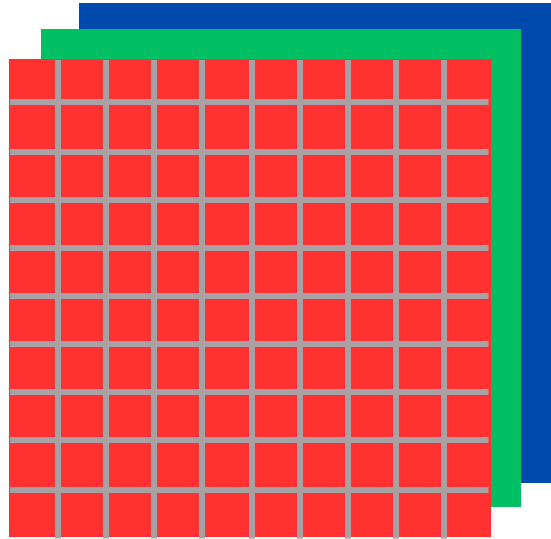
CMY

CMYK

YIQ

HSV

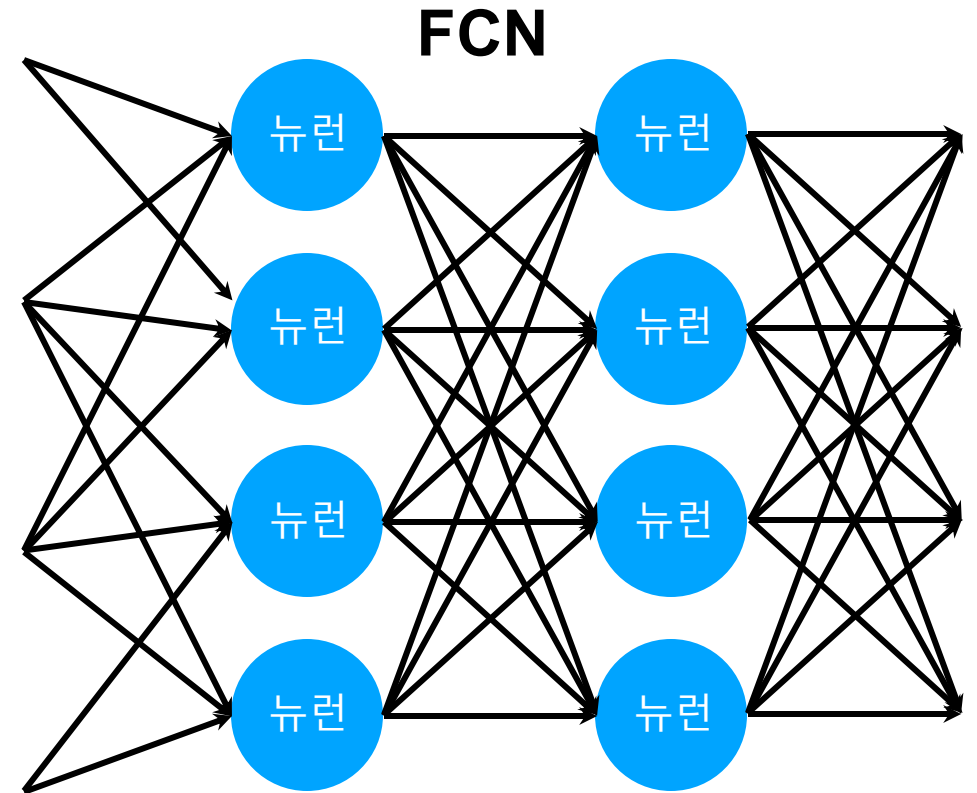
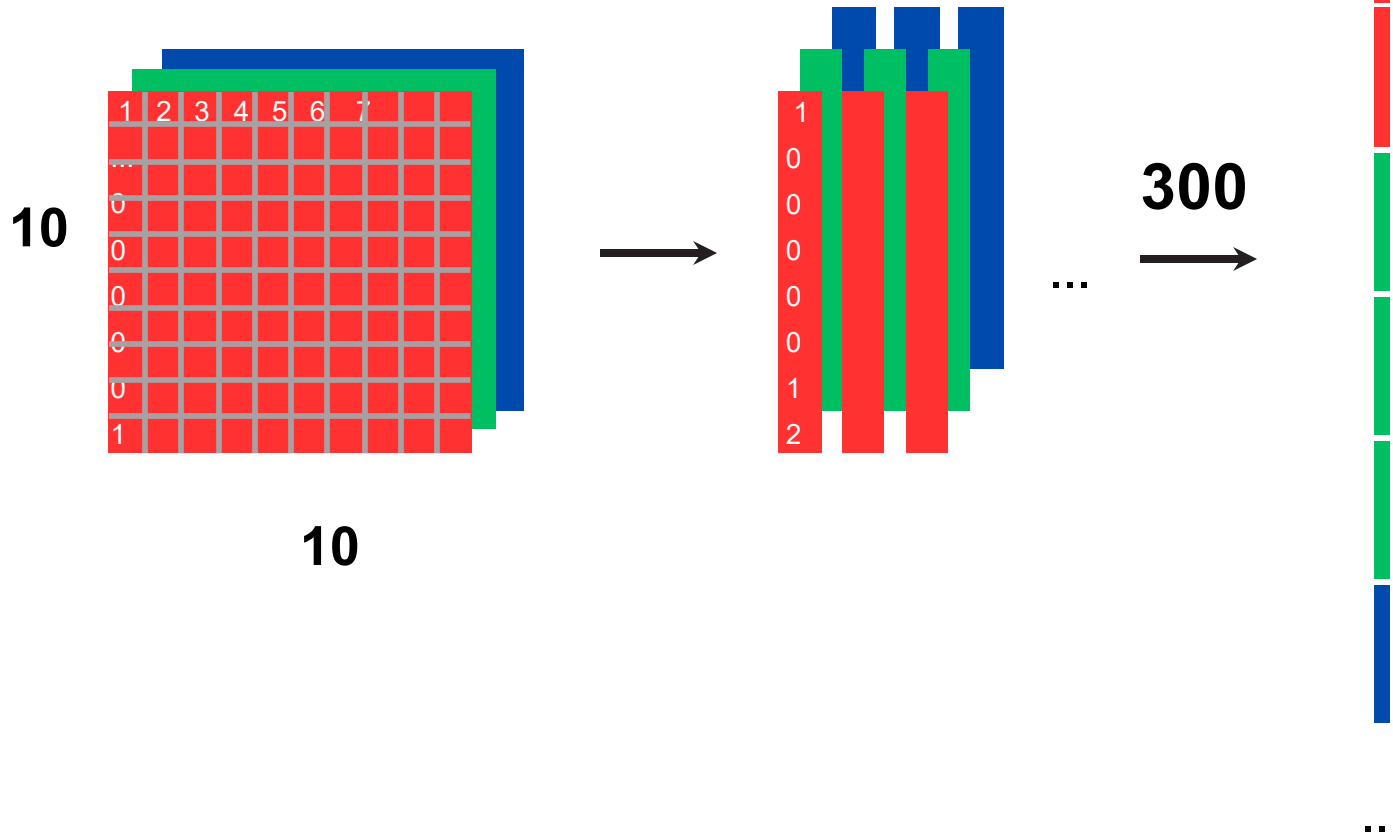
HLS



- Red, Green, Blue 세 채널로 표현
- 이미지 크기 = 가로 x 세로 x 채널 수
(Width x Height x Channel)

FCN과 이미지

이미지 데이터를 Fully Connected Network로 학습하려면?



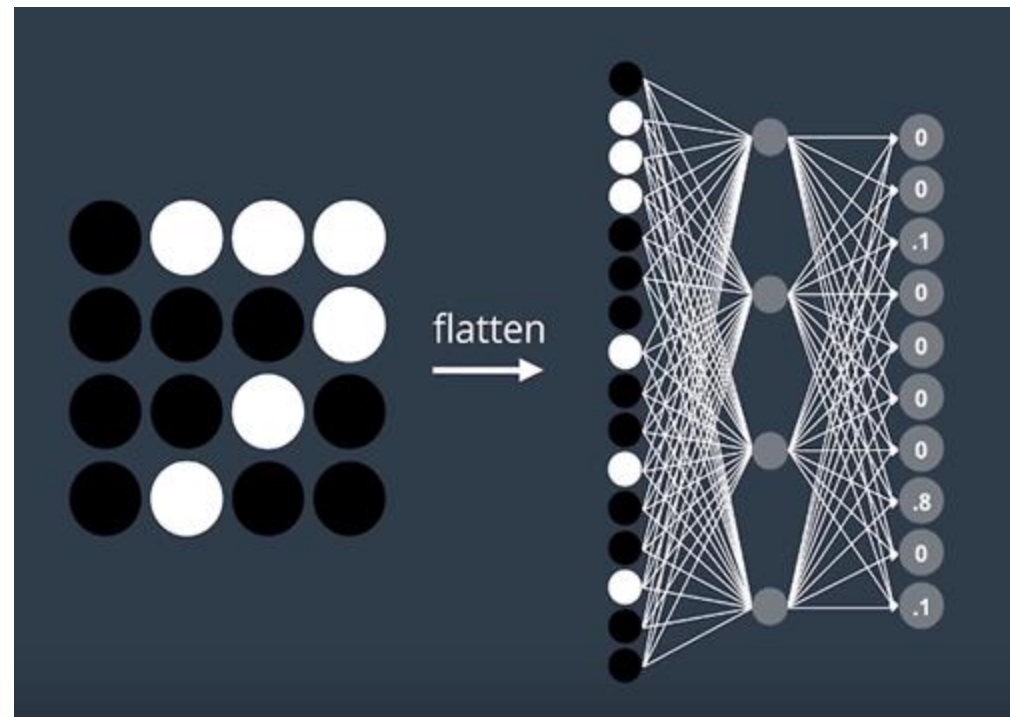
FCN으로는 한계가 있다

1. 파라미터 수 증가로 인한 문제

- 모든 픽셀을 연결하기 때문에 파라미터 수가 폭발적으로 증가
 - 예: 100x100 이미지 → 1만 픽셀
 - 첫 은닉층을 1천 노드로 설정 시 1천만 개의 연결 필요
- 결과:
 - 과적합 위험 → 일반화 성능 저하
 - 계산 비용 증가 → 학습 및 추론 속도 저하
 - 확장성 낮음 → 고해상도 이미지 처리에 비효율적
(예: 1024x1024x3 이미지 → 3백만 입력 노드 필요)

2. 지역 정보 손실(Local Information Loss)

- 픽셀 간 공간적 구조(예: 인접 픽셀의 관계) 반영 불가
- 이미지를 1차원 벡터로 변환 → 패턴(예: 가장자리, 모서리) 학습 어려움



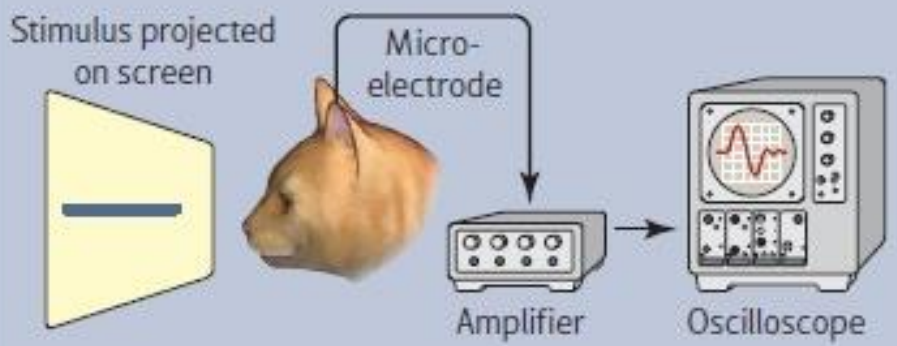
구분	한계점	구체적 이유·영향
파라미터 폭증	입력 $H \times W \times C$ 전체를 한 벡터로 펼쳐 곧바로 Dense → 노드 수가 수십 만~수백 만, 파라미터는 수억 개까지 급증	• 학습·추론 메모리와 연산량이 폭발 • 작은 데이터셋에서는 심각한 과적합
공간 구조 정보 손실	픽셀을 1-D로 flatten하면 이웃 관계·위치 정보가 사라짐	• 모델이 3×3 에지나 패턴 같은 지역 특징(local feature) 를 학습하지 못함 • 같은 물체라도 위치가 조금만 달라져도 출력을 다시 학습해야 함
변환(이동) 불변성 부족	파라미터가 픽셀 “절대 위치”에 묶여 있어 translation invariance 가 없음	• 객체가 화면에서 한 칸만 이동해도 완전히 다른 입력으로 인식 • 데이터 증강으로 해결하려면 샘플 수가 기하급수적으로 필요
학습 안정성	깊어질수록 표준 MLP는 기울기 소실·폭발 문제가 심해짐	• CNN·Residual·BN 같은 구조적 안전장치 부재
전역만 보는 수용영역	입력 전체가 한꺼번에 연결 → 뉴런마다 Receptive Field 가 이미지 전체	• 세밀한 국소 패턴을 점진적으로 조합하는 계층적 특징 표현 이 불가능
재사용 불가 가중치	같은 필터(가중치)를 여러 위치에 공유하지 못함	• 동일 패턴(예: 수직 에지)을 화면 곳곳에서 새 파라미터로 반복 학습 해야 함
확장성·전이학습 한계	입력 해상도가 달라지면 FC 층 크기도 완전히 재설계·재학습 필요	• 224×224에서 학습한 가중치를 256×256엔 그대로 쓸 수 없음
시각화·해석 어려움	가중치 매트릭스가 픽셀 위치와 직접 매핑되지 않아 feature map 직관 부족	• 학습된 “무엇을 보고 결정을 내렸는지” 설명력이 떨어짐

사람이 어떻게 이미지를 인식하는가?

4.41 과학적 방법: 시각 피질의 개별 세포는 다양한 자극에 어떻게 반응하는가?

Method

1. 마취된 고양이의 한쪽 눈을 열어 다양한 방향의 선과 같은 일련의 시각 자극이 망막의 특정 부위로 향하도록 했습니다.
2. 시각 피질에 미세 전극을 삽입하여 개별 세포가 선에 반응할 때의 발화율을 관찰했습니다.



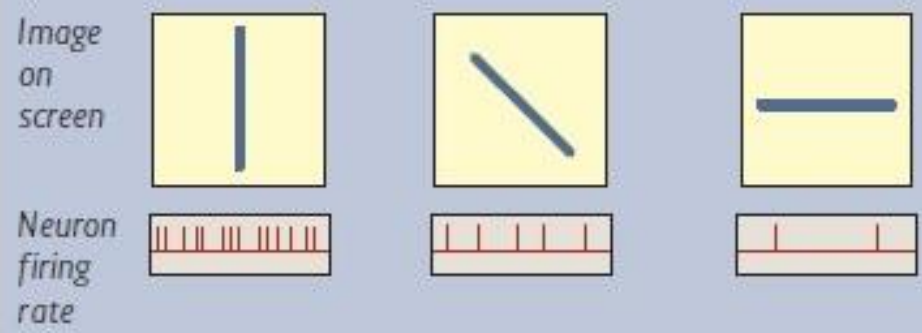
3. 세포가 발화하면 신경 자극이 증폭되어 오실로스코프에 표시되었습니다. (이 절차는 여러 개별 세포의 반응을 관찰하기 위해 반복되었습니다.)

Results

일부 세포는 수직선 자극에 더 빠르게 반응했습니다

수직 선호 뉴런은 기울어진 선에 대해 중간 정도의 속도로 발화했습니다

이러한 세포는 수평선 자극에는 전혀 발화율이 증가하지 않았습니다



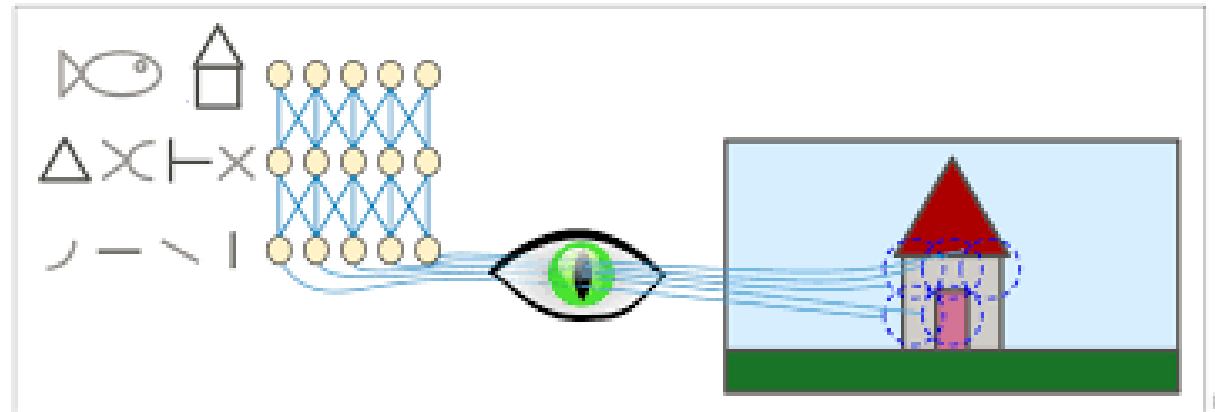
결론

시각 피질의 각 뉴런은 특히 빠르게 반응하는 "목표" 자극을 가지고 있습니다. 이러한 목표 자극에는 특정 방향의 선이나 호와 같은 저수준의 특징이 포함됩니다.

사람이 어떻게 이미지를 인식하는가?

Hubel과 Wiesel의 연구:

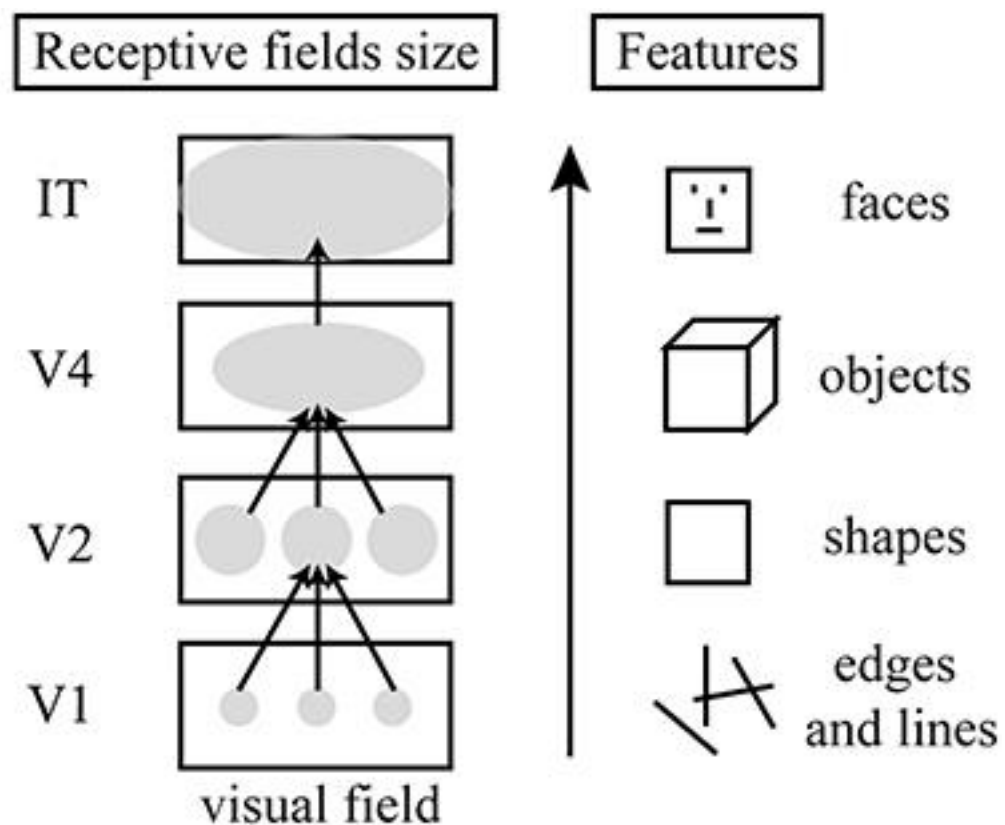
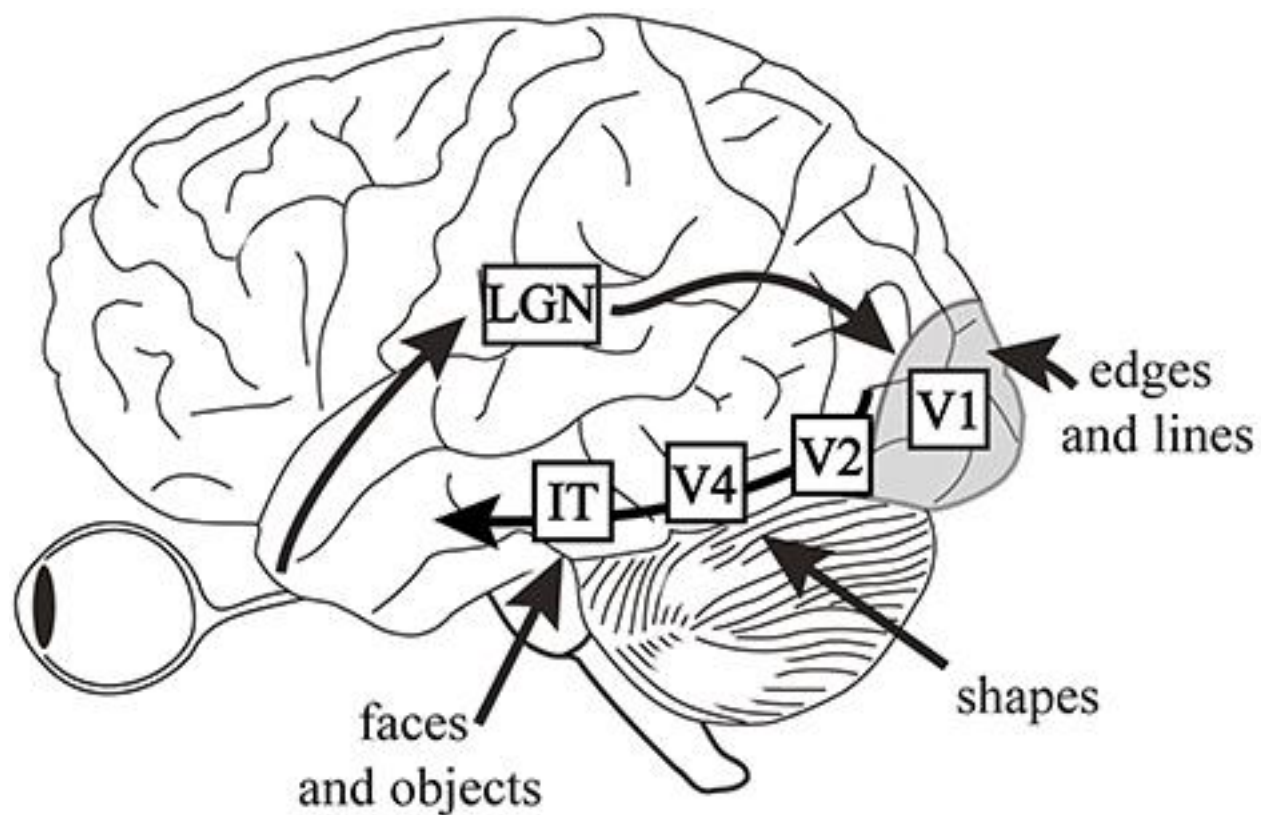
- 고양이의 시각 피질 실험에서 고양이 시야의 한쪽에 자극을 주었더니 전체 뉴런이 아닌 특정 뉴런이 활성화되는 것을 발견
- 시각 피질 안의 많은 뉴런이 작은 국부 수용장(Local Receptive Field)을 가진다.
 - 뉴런들이 시야의 일부 범위 안에 있는 시각 자극에만 반응
 - 뉴런 수용장들은 서로 겹칠 수 있어 이를 합치면 전체 시야를 감싸게 된다.
- 어떤 뉴런(같은 수용장에서)은 수평선의 이미지에만 반응하는 반면 다른 뉴런은 다른 각도의 선에 반응한다
- 어떤 뉴런은 큰 수용장(Receptive Field)를 가져서 저수준(low-level) 패턴이 조합된 더 복잡한 패턴에 반응한다



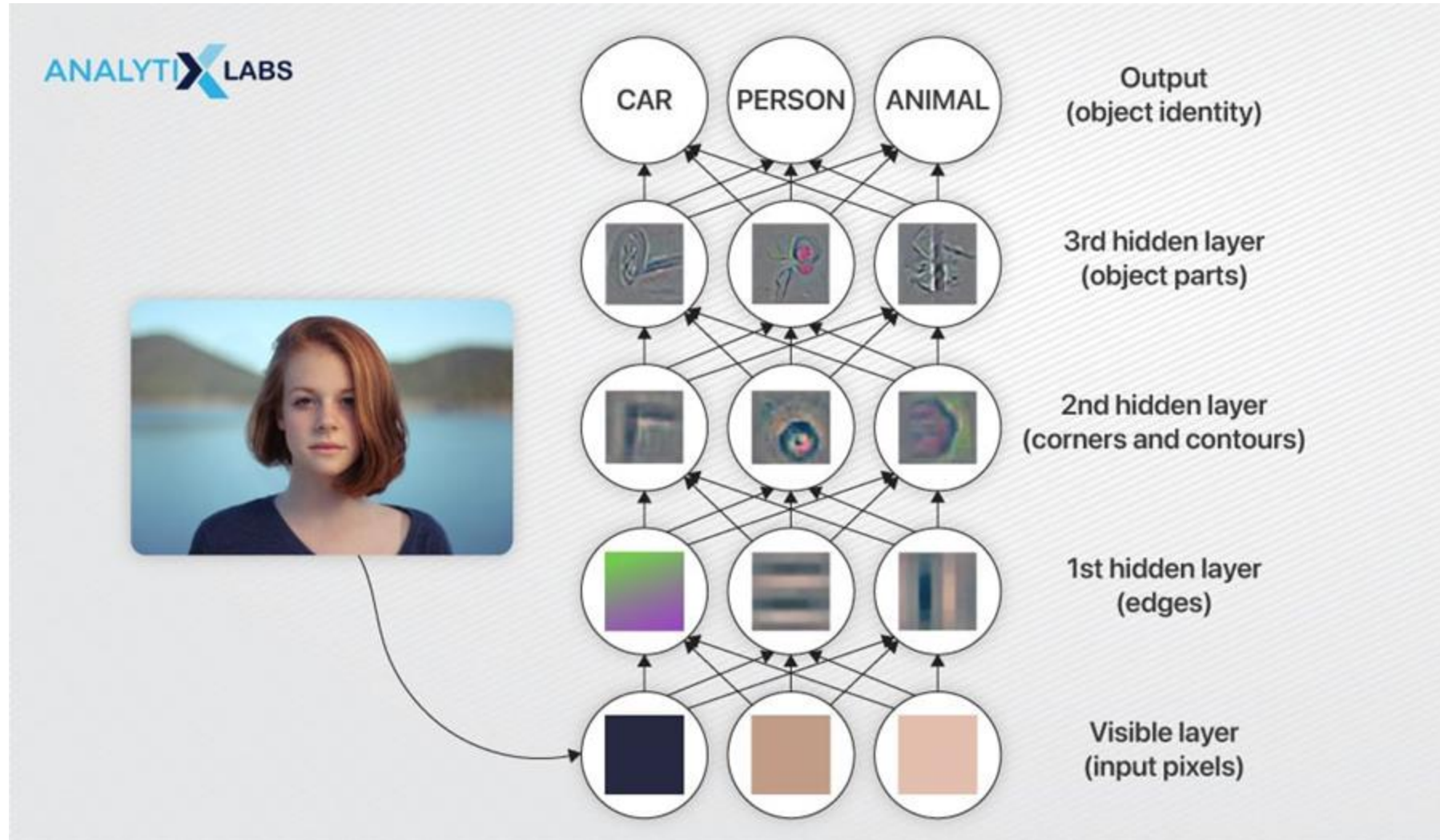
핵심 아이디어	연구에서 관찰한 내용	왜 CNN 공부에 중요한가?
단순세포 & 복합세포	- 단순세포 : 화면의 한 조그만 사각형 안에서 직선 이 보이면 반응. - 복합세포 : 같은 직선을 화면 어디에 두어도 반응.	CNN도 처음 층에서 직선(에지) 를 찾고, 뒤 층으로 갈수록 위치에 덜 민감해지는 과정을 닮음.
방향 선호(Orientation)	어떤 뉴런은 45° 기울어진 선에만, 다른 뉴런은 가로선에만 강하게 반응.	CNN 안의 필터도 학습이 되면 ‘세로선 필터’, ‘가로선 필터’처럼 특정 방향 을 잘 잡아냄
수용장(Receptive Field)	뉴런 하나가 보는 영역이 아주 작음 → 층을 따라 올라갈수록 점점 넓어짐.	CNN에서 3×3 같은 작은 필터를 많이 쌓으면, 최종적으로 큰 영역 을 볼 수 있게 되는 원리와 같음
컬럼 구조	비슷한 방향을 좋아하는 뉴런끼리 기둥 모양으로 모여 있음(orientation column).	CNN에서 가중치를 공간 전체에 공유 해 같은 패턴을 반복해서 찾는 아이디어에 영감을 줌.
두 눈 우세(Left/Right eye)	어떤 뉴런은 왼쪽 눈, 어떤 뉴런은 오른쪽 눈 자극에 더 잘 반응.	입력 채널(R·G·B 등)을 따로 받았다가 뒤에서 합치는 CNN 구조와 비슷한 개념.
성장기 가소성(Critical Period)	아기 고양이는 어린 시절에만 시각 회로가 쉽게 바뀜.	초기 학습(critical period/훈련)이 회로·가중치의 기본 틀을 만들고 그 시기를 지나면 대규모 구조 변화가 어려워지고, 미세 조정만 가능함
하이퍼컬럼	한 지점에 대해 ‘0–180° 직선’ + ‘왼눈·오른눈’ 정보가 한 세트로 존재.	CNN 한 픽셀에 여러 채널(필터 결과)을 겹쳐 저장 하는 구조와 닮음.
노벨상 (1981)	“시각 정보 처리 원리” 발견으로 생리·의학 부문 수상.	뇌 연구가 컴퓨터 비전 발전 에 직접 씨앗이 되었음을 상징적으로 보여 줌.

사람이 어떻게 이미지를 인식하는가?

* LGN은 Lateral Geniculate Nucleus의 약자로, 한국어로는 외측슬상핵이라고 합니다. 이것은 시각 정보를 뇌로 전달하는 중요한 역할을 하는 뇌의 부분입니다.



사람이 어떻게 이미지를 인식하는가?



LeNet-5

LeNet-5는 CNN의 가능성을 처음으로 실증한 모델로, 이미지 인식 분야의 새로운 장을 열었습니다

- 발표 : 얀 르쿤(1998)
- 목적 : 손글자 숫자 인식(MNIST 데이터 셋)
- 구조 : 합성곱 신경망(CNN)의 최초 실용적 구현
- http://vision.stanford.edu/cs598_spring07/papers/Lecun98.pdf

위 논문에서는 우리에게 익숙한 완전 연결층(FCN), 시그모이드 활성화 함수 같은 구성요소와 함께, 합성곱 층(CNN Layer)와 풀링 층(Pooling Layer)라는 새로운 구성요소가 등장하였습니다.

얀 르쿤 (Yann LeCun): 컴퓨터 비전 분야의 선구자로, 합성곱 신경망 (Convolutional Neural Network) 알고리즘을 개발하여 이미지 및 비디오 처리 기술 발전에 크게 기여했습니다. 현재 뉴욕대학교 교수이자 메타 (Meta)의 최고 AI 과학자입니다.

CNN(Convolutional Neural Network)

CNN의 특징

1. 부분 연결(Partial Connectivity)

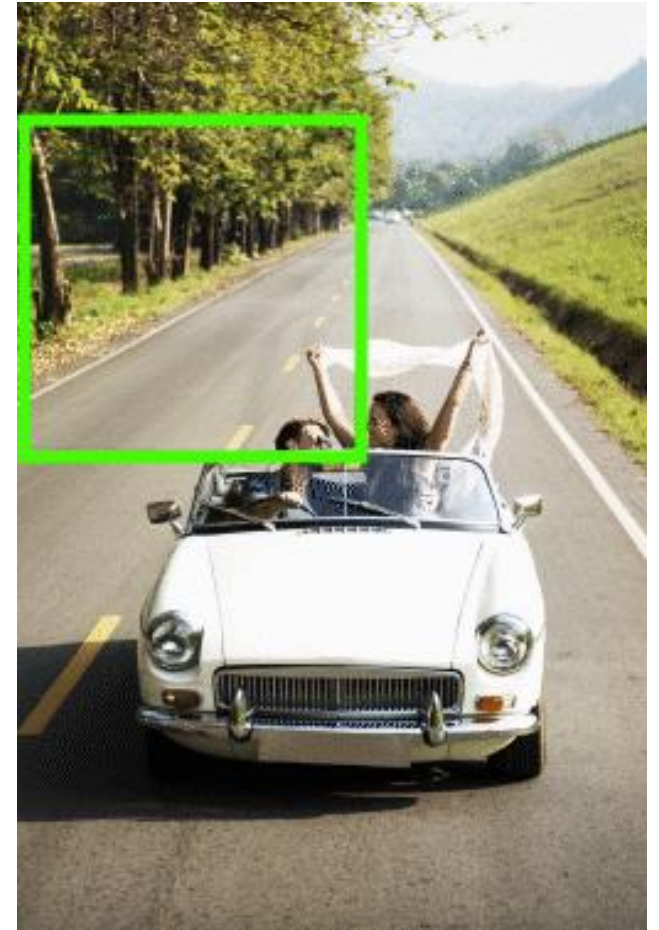
- 완전 연결층의 모든 노드 연결 방식을 대체
- 전체 이미지를 한번에 분석하지 않고, 작은 지역(근접 픽셀 간의 관계)부터 점진적 학습
- 이미지의 **지역적 정보(local information)**를 효과적으로 학습

2. 가중치 공유(Weight Sharing)

- 동일한 필터(커널)를 이미지 전체에 적용
- 파라미터 수를 크게 줄여 **계산 효율성을 향상**

3. 계층적 특징 표현

- 입력과 가까운 하위 레이어에서 단순한 특징(에지, 선 등)을 학습
- 출력에 가까운 상위 레이어에서 추상적이고 복잡한 특징(물체의 부분 등)을 학습
- 저수준에서 고수준으로 계층적 특징 표현이 가능



CNN(Convolutional Neural Network)

핵심 특징	의미	중요한 점
부분 연결(Local Connectivity)	필터(커널)가 입력의 **작은 패치(예: 3 × 3)** 에만 연결되어 값을 곱-합	이웃 픽셀 간 패턴(에지·코너)을 효율적으로 학습하며, 파라미터 수를 대폭 줄임
가중치 공유(Weight Sharing)	같은 필터 파라미터를 전체 공간 에 슬라이딩 적용	화면 어디에 있든 같은 패턴을 한 세트 가중치로 탐지 → 메모리·학습 샘플 요구량↓
계층적 특징 추출(Hierarchical Representation)	얕은 층: 선·에지 → 중간 층: 질감·모서리 → 깊은 층: 객체 부분	복잡한 구조를 단순-고차 패턴 단계별로 조합, 설명력 및 전이학습 성능↑
수용장(Receptive Field) 확장	층이 깊어질수록 한 뉴런이 보는 영역이 넓어짐	작은 필터를 쌓기만 해도 전역 문맥 파악 가능; 이미지 크기 변화에 강인
변환 공차(Translation Equivariance & Invariance)	Conv → 위치가 바뀌면 출력도 동일하게 이동 (equivariance) Pool → 위치 변화에 둔감 (invariance)	객체가 화면에서 조금 움직여도 인식을 유지, 데이터 증강 부담↓
파라미터 효율·계산 병렬화	FC 대비 파라미터·FLOPs 수십 배↓, 텐서 연산은 GPU에 최적	딥 네트워크를 실용적인 시간 안에 학습·추론 가능하게 함
자동 특징학습(End-to-End Learning)	별도 특성 공학 없이 이미지 → 레이블 직접 매핑	전통 CV(핸드-크래프트) 대비 성능·확장성 우위
시각화·해석 가능	Feature map·Grad-CAM으로 무엇을 보고 결정했는지 확인	신뢰성·디버깅·모델 설명(Explainable AI) 기반 제공

Convolution 연산

Convolution Filter를 이용한 연산



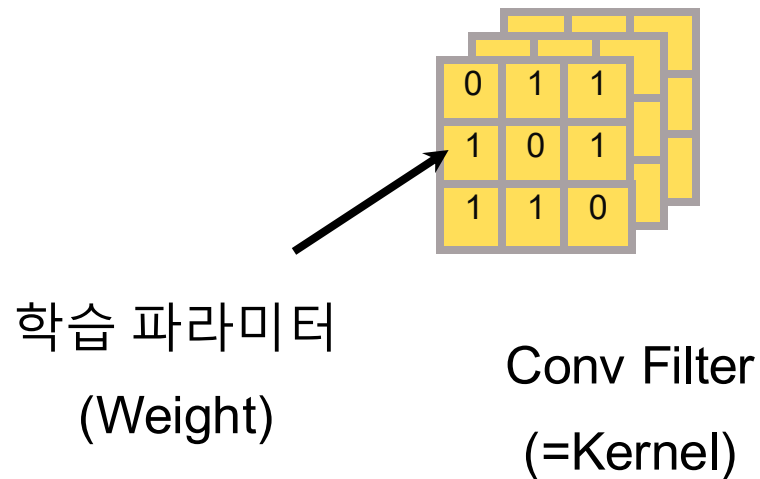
Convolutional Layer



Convolutional Neural Network

Convolution Filter

지역적 특성을 추출할 수 있는 단위

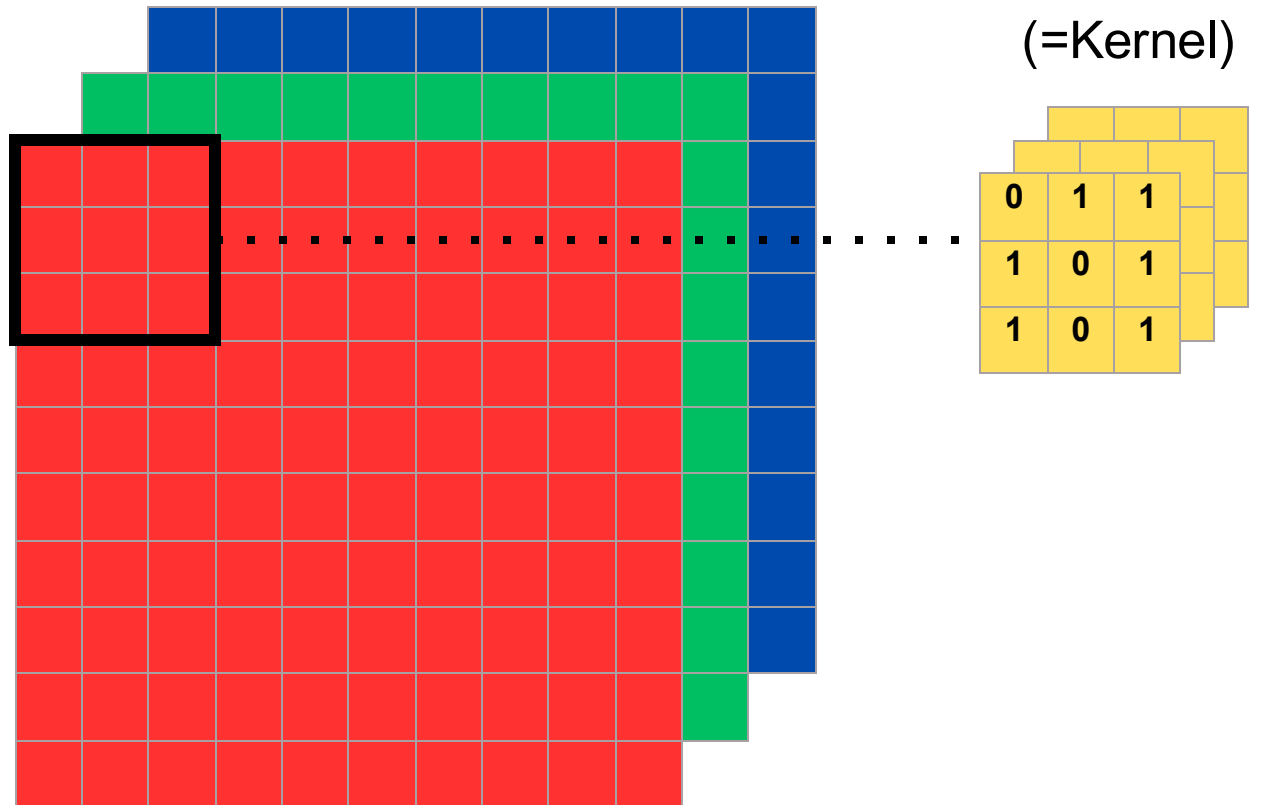


- 이미지 크기보다 작은 행렬
- 이미지에서 Conv Filter 행렬 크기만큼의 영역에 대해 계산을 수행하여 이미지 내의 패턴이나 특징을 인식

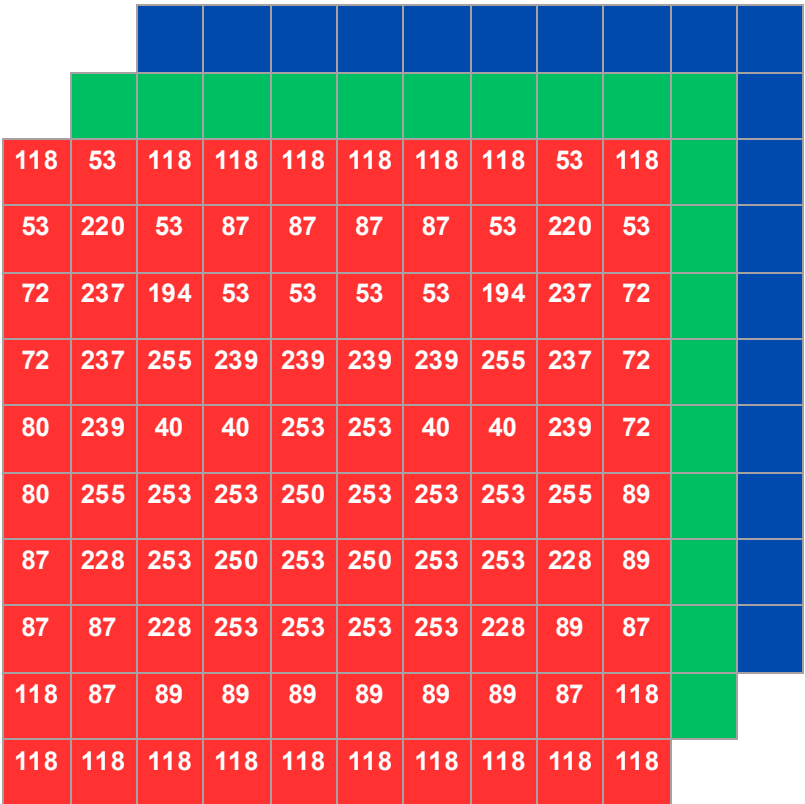
Convolutional Layer

컨볼루션 연산이 수행되는 레이어
지역적 특성을 추출하는 과정

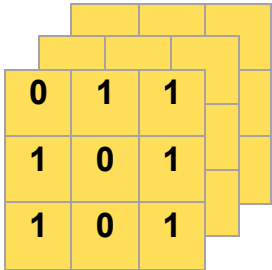
Convolution



Convolution



Conv Filter
(=Kernel)



Convolutional Layer

118	53	118	118	118	118	118	118	53	118				
53	220	53	87	87	87	87	53	220	53				
72	237	194	53	53	53	53	194	237	72				
72	237	255	239	239	239	239	255	237	72				
80	239	40	40	253	253	40	40	239	72				
80	255	253	253	250	253	253	253	255	89				
87	228	253	250	253	250	253	253	228	89				
87	87	228	253	253	253	253	228	89	87				
118	87	89	89	89	89	89	89	87	118				
118	118	118	118	118	118	118	118	118	118				

Conv Filter
(=Kernel)

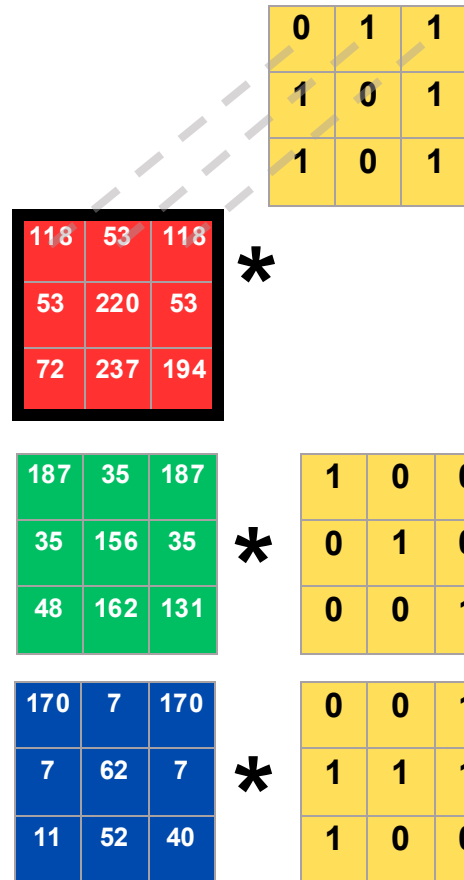
0	1	1							
1	0	1							
1	0	1							

118	53	118							
53	220	53							
72	237	194							

*

$$\begin{aligned} &118 \times 0 + 53 \times 1 + 118 \times 1 + \\ &53 \times 1 + 220 \times 0 + 53 \times 1 + \\ &72 \times 1 + 237 \times 0 + 194 \times 1 \end{aligned} = 543$$

Convolutional Layer



$$\begin{aligned} &118 \times 0 + 53 \times 1 + 118 \times 1 + \\ &53 \times 1 + 220 \times 0 + 53 \times 1 + \\ &72 \times 1 + 237 \times 0 + 194 \times 1 \end{aligned}$$

543

+

$$\begin{aligned} &187 \times 1 + 35 \times 0 + 187 \times 0 + \\ &35 \times 0 + 156 \times 1 + 35 \times 0 + \\ &48 \times 0 + 162 \times 0 + 131 \times 1 \end{aligned}$$

474

+

$$\begin{aligned} &170 \times 0 + 7 \times 0 + 170 \times 1 + \\ &7 \times 1 + 62 \times 1 + 7 \times 1 + \\ &11 \times 1 + 52 \times 1 + 40 \times 0 \end{aligned}$$

257

+

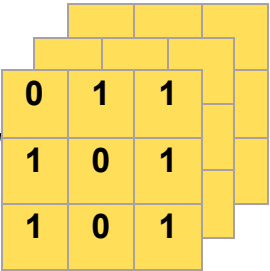
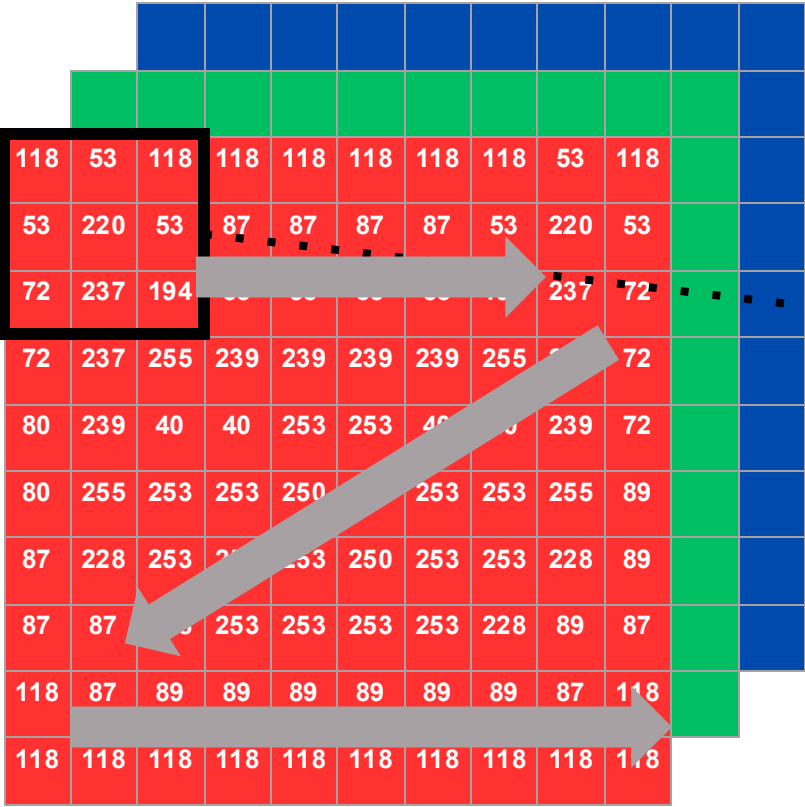
3
Bias

=

1277

Convolutional Layer

입력값



출력값 = 특성 맵(Feature Map)

예: 외곽 선 특성 정보

1274	1378	1530	1532	1532	1593	1384	1220
1365	1668	1436	1389	1389	1409	1744	1294
1316	1679	1744	1640	1572	1754	1883	1284
1649	2069	2215	2676	2685	2322	2099	1526
1795	2341	2839	2857	2907	2694	2532	1770
2422	2819	3138	3273	3066	3217	2741	2251
1909	2298	2518	2467	2557	2419	2249	1729
1974	1755	1820	1845	1845	1806	1608	1863

Convolutional Layer

출력값 = 특성 맵(Feature Map)

예: 외곽 선 특성 정보

1274	1378	1530	1532	1532	1593	1384	1220
1365	1668	1436	1389	1389	1409	1744	1294
1316	1679	1744	1640	1572	1754	1883	1284
1649	2069	2215	2676	2685	2322	2099	1526
1795	2341	2839	2857	2907	2694	2532	1770
2422	2819	3138	3273	3066	3217	2741	2251
1909	2298	2518	2467	2557	2419	2249	1729
1974	1755	1820	1845	1845	1806	1608	1863

(1,1) 주변 3x3 영역에서 지역적인 특성을 수치화

(1,2) 주변 3x3 영역에서 지역적인 특성을 수치화

...

Strided Convolution

Input Image

1	9	8	4	4	5	7
4	8	6	7	9	1	7
4	0	5	9	3	8	4
7	3	5	9	0	5	4
7	4	1	1	8	1	2
7	6	6	9	8	7	6
3	6	3	5	4	2	7

Kernel

0	-1	0
-1	5	-1
0	-1	0

Feature Map

*

=

이 과정의 핵심은 바로
“Kernel”

Kernel

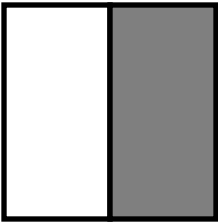
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

*

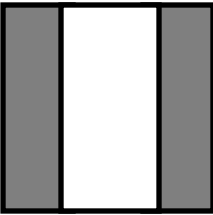
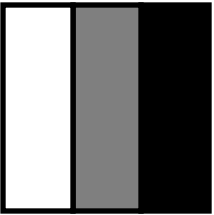
1	0	-1
1	0	-1
1	0	-1

=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0



*



Kernel

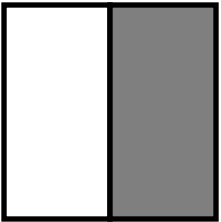
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

*

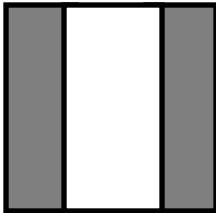
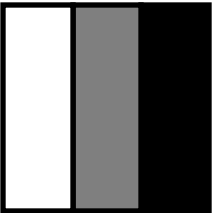
1	0	-1
1	0	-1
1	0	-1

=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0



*

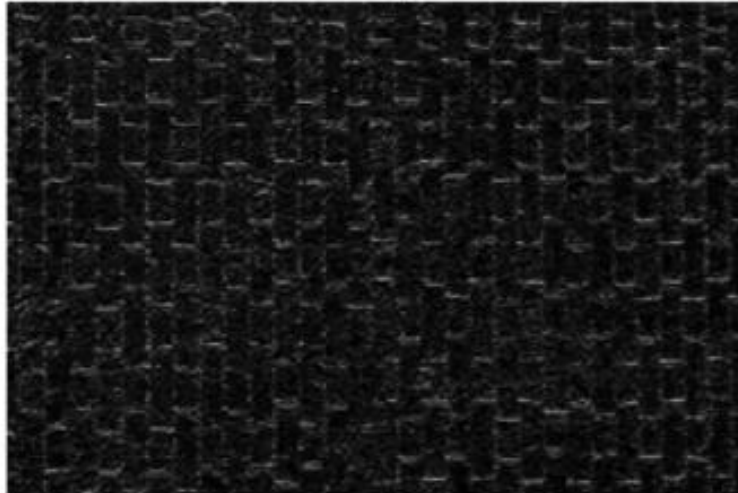


Kernel

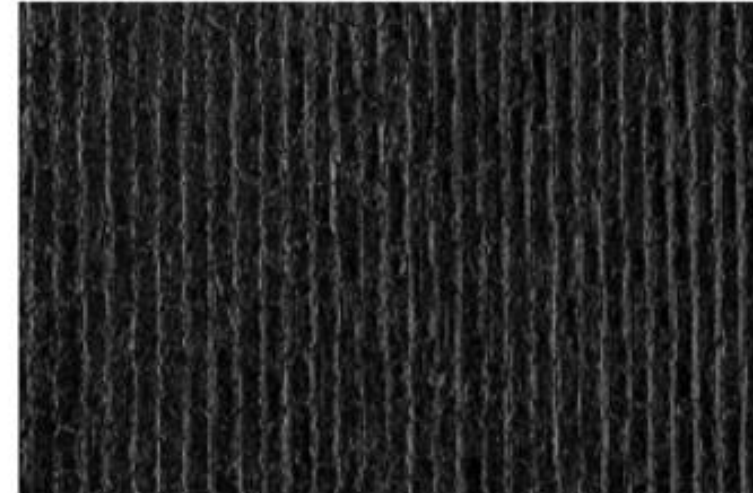
original image



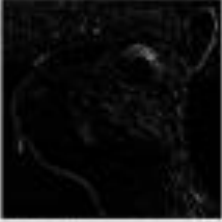
horizontal edge detection



vertical edge detection



Kernel

Operation	Filter	Convolved Image
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	

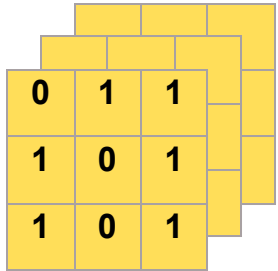
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

Kernel



Input

Conv Filter (=Kernel)



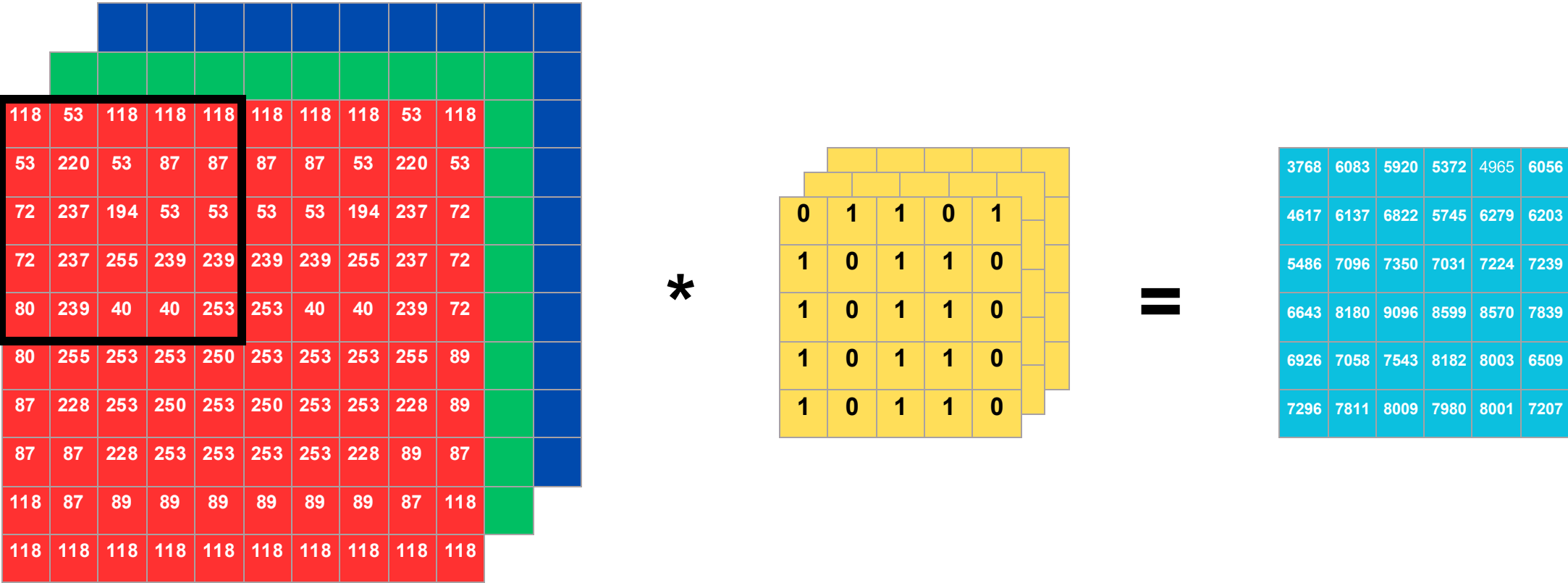
0	1	1
1	0	1
1	0	1

- 필터란 무엇인가?
 - 작은 크기의 행렬로, 이미지의 특징(feature)을 추출하는 핵심 도구
 - 이미지의 국소적 영역(local region)에 적용됨
- 필터의 주요 기능
 - 이미지의 지역적 정보를 추출하여 다양한 패턴 식별 가능
예) 에지(edge), 텍스처(texture), 색상 변화 등
 - 각 필터는 서로 다른 특징을 학습하여 이미지 분석
- 채널
 - 색상이 있는 이미지는 RGB 채널 각각에 필터를 적용
 - 각 채널은 다른 가중치(weight)를 가지며 독립적으로 처리됨
- 특징 맵(Feature Map)
 - 필터를 통과한 데이터는 특징 맵 또는 활성화 맵(Activation Map)이라고 불림
 - 특징 맵은 필터가 감지한 특정 패턴을 강조한 결과물

커널 크기 (Kernel Size)

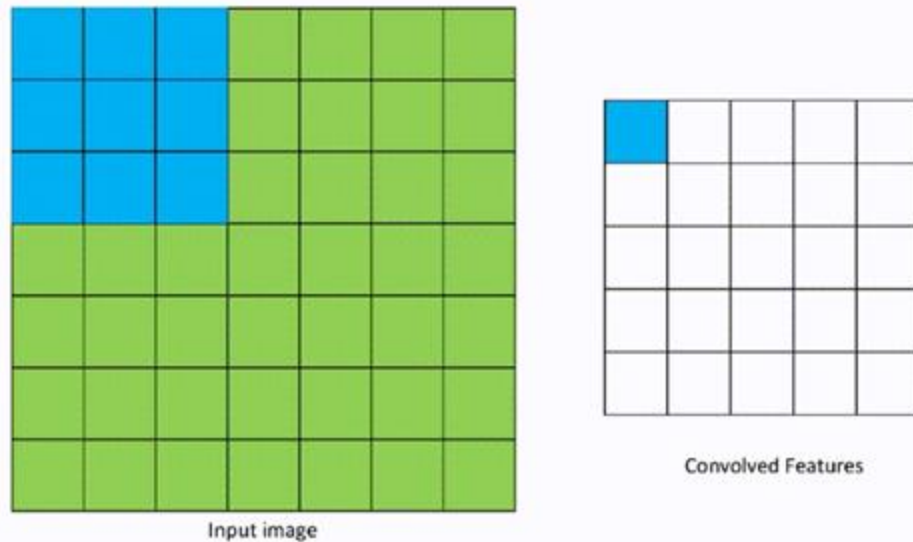
커널 크기(Kernel Size)는 CNN에서 사용하는 필터(Filter)의 행렬 크기를 의미

- 3x3, 5x5, 7x7 등

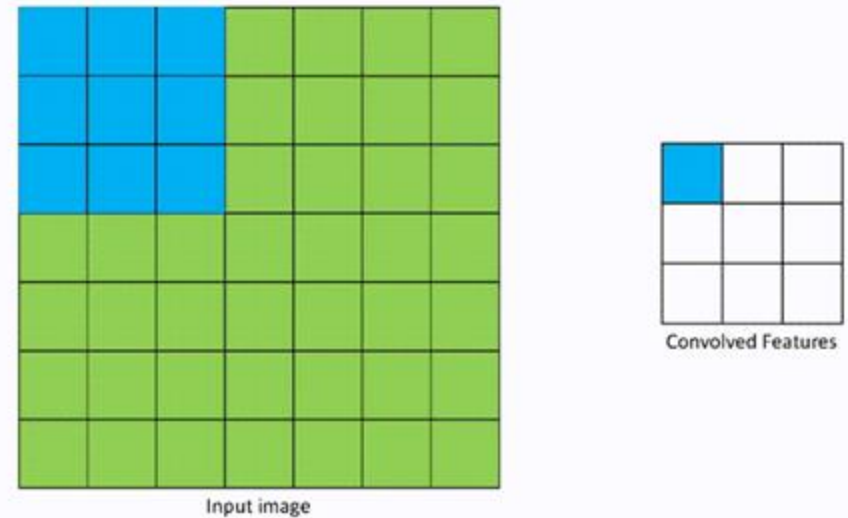


Stride

필터의 이동 간격, 즉 필터를 몇 칸씩 이동할 지 설정하는 것
스트라이드 크기가 커지면 출력 특징 맵의 크기는 작아짐 (보통 스트라이드 크기는 1 또는 2로 설정)

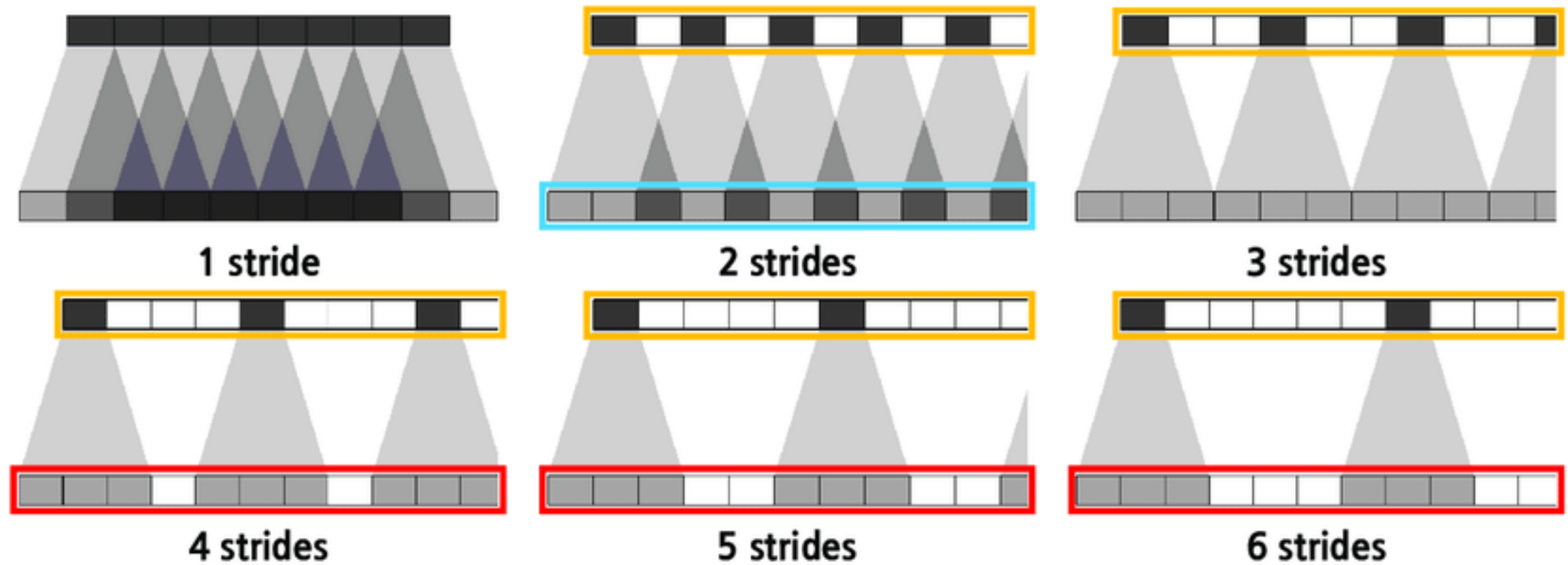


[1, 1] stride



[2, 2] stride

Stride

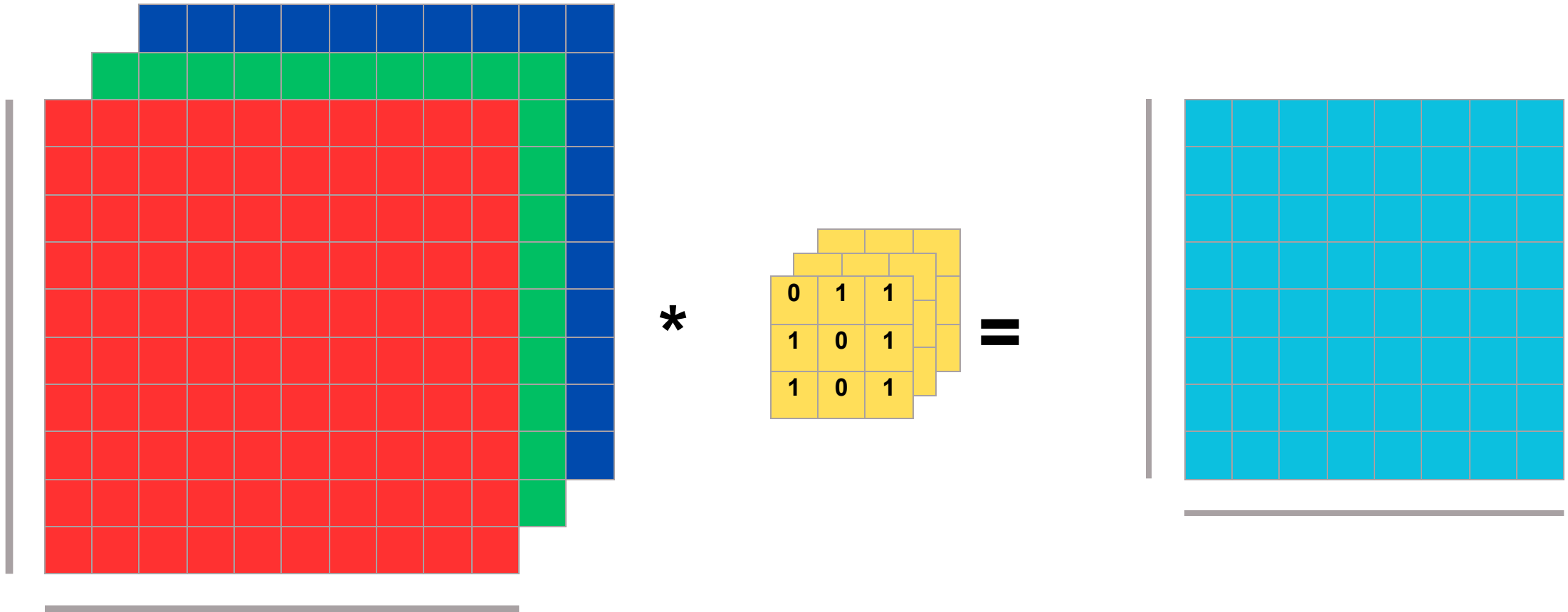


Stride의 직관적 의미

- Convolution은 주변 정보를 모은다
- 보통은 Stride로 건너뛰어도, 아예 건너뛸 픽셀은 없다
- Convolution 연산의 복잡도를 줄임
- 너무 큰 stride는 중요한 정보가 누락될 위험이 있다

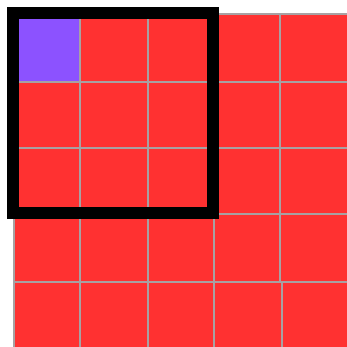
특징 맵(Feature Map)의 해상도가 줄어들며
공간적인 세밀함(Spatial Resolution)이 감소한다

Feature Map

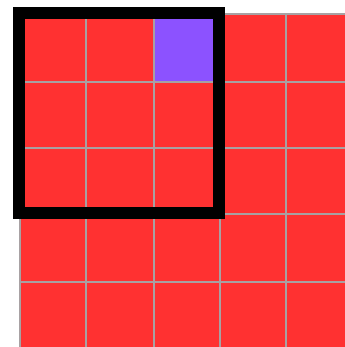


Input Image Size == Feature Map Size ??

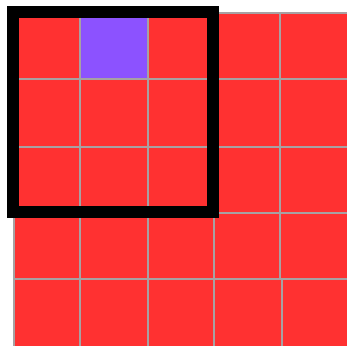
Padding



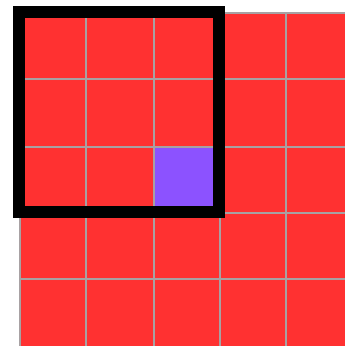
1번 사용
됨



3번 사용
됨



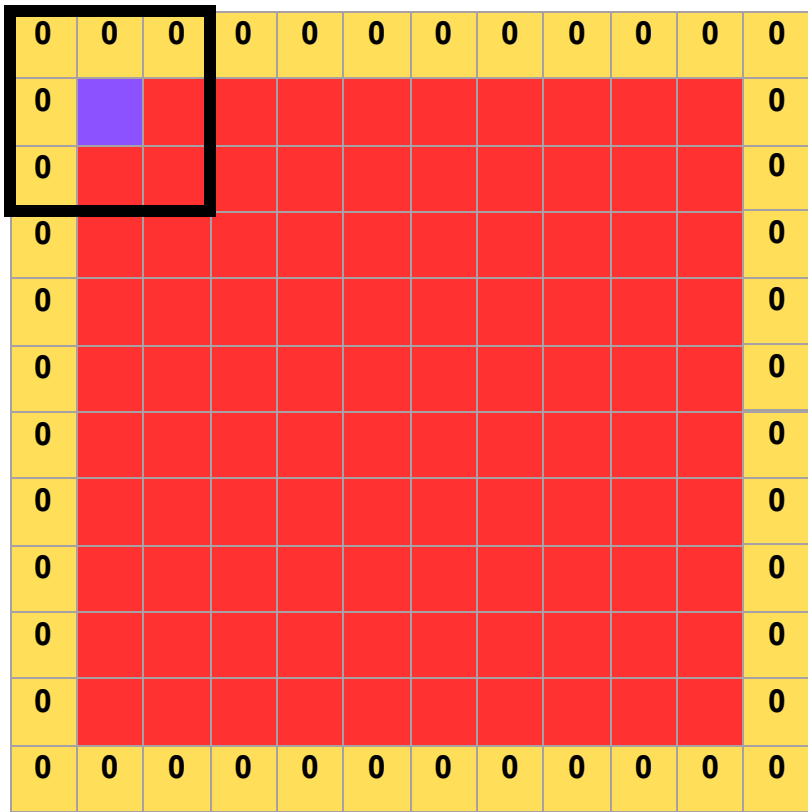
2번 사용
됨

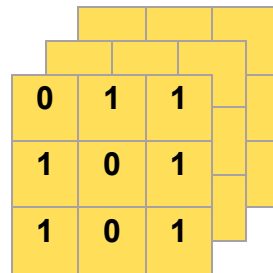


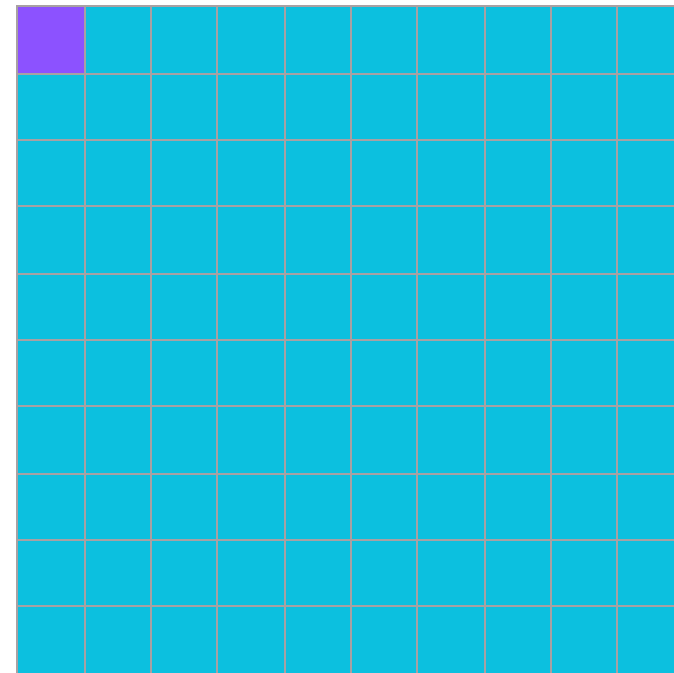
9번 사용
됨

가장자리일수록 특성에 반영되는 횟수가 적음

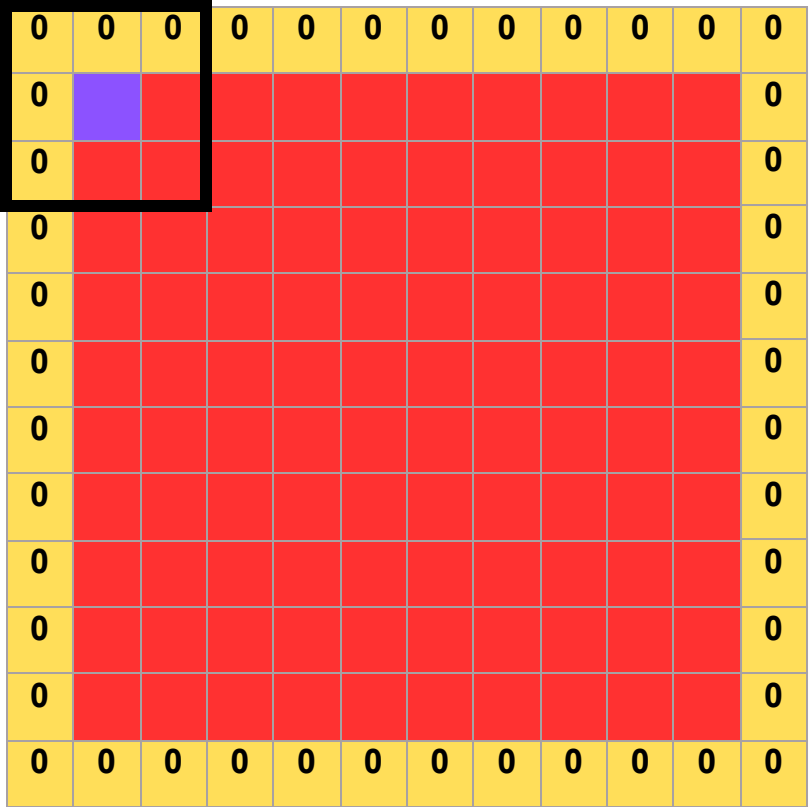
Padding







Padding



1~9번



4~9번

Padding

- 입력 데이터의 가장자리에 0(또는 다른 값)을 추가하여 입력 크기를 확장하는 과정
- 목적
 - 필터 연산 시 이미지 가장자리 정보 손실 방지
 - 출력 크기를 조정하여 원하는 네트워크 구조 유지
- 패딩의 크기는 필터(커널)의 크기와 스트라이드에 따라 조절될 수 있음

Padding

1. Valid Padding (패딩 없음)

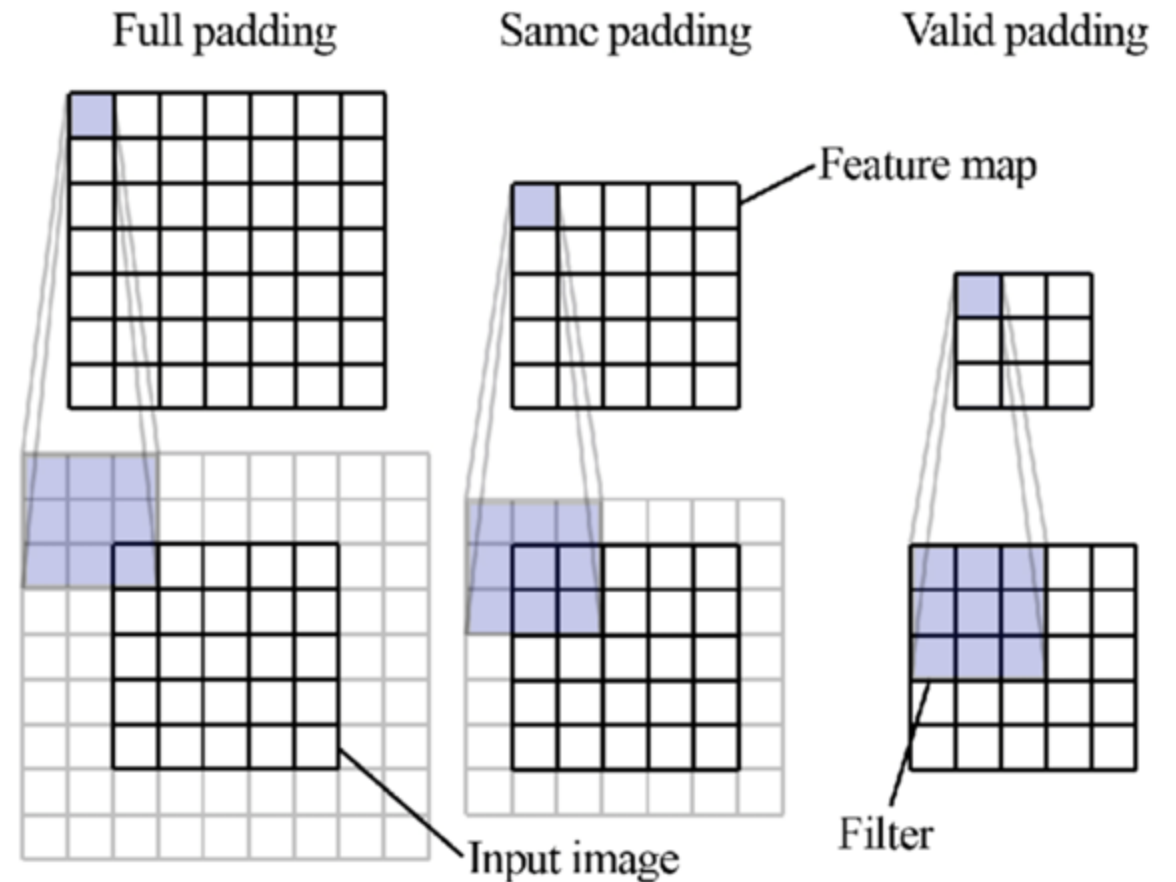
- 패딩을 사용하지 않음 (필터가 적용될 수 있는 영역만 사용)
- 출력 크기가 줄어듦
- 계산량 감소, 간단한 특징 학습에 유리

2. Same Padding

- 입력 크기를 유지하기 위해 필요한 만큼 패딩 추가
- 필터를 이미지 전체에 적용 → 출력 크기 = 입력 크기
- CNN 모델에서 가장 널리 사용

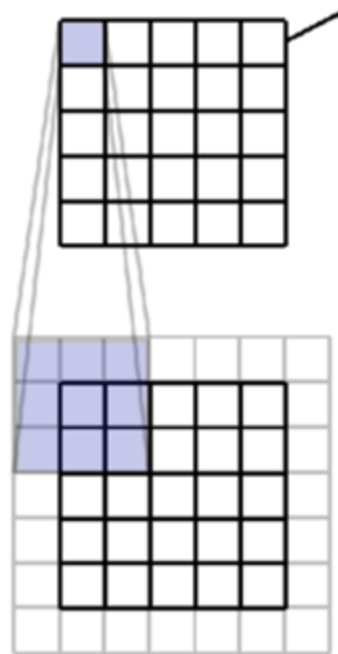
3. Full Padding

- 출력 크기가 입력 크기보다 커지도록 패딩을 추가
- 더 넓은 컨볼루션 영역을 학습할 때 사용

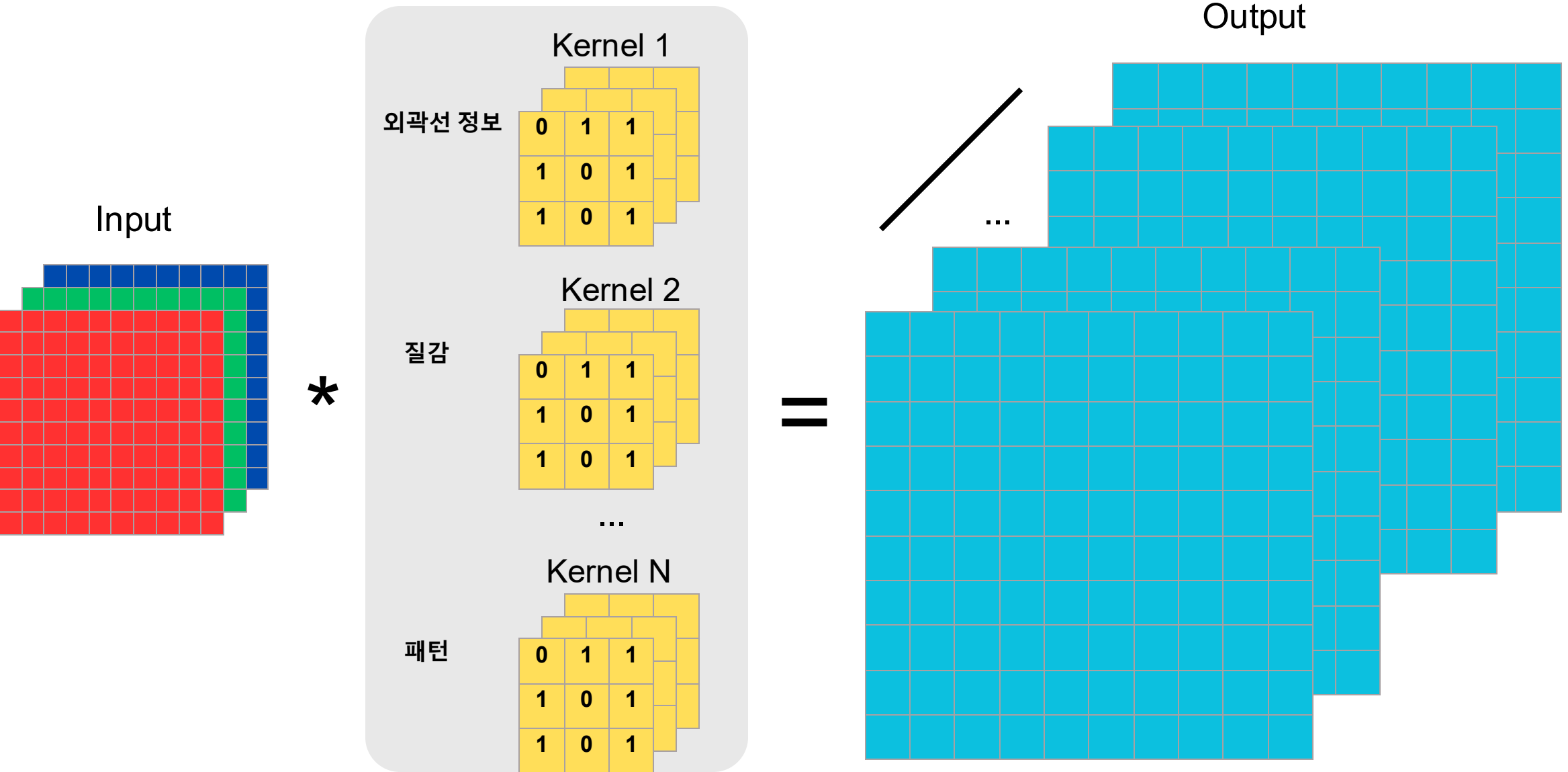


$$\text{Output Size} = \frac{\text{Input Size} - \text{Filter Size} + 2 \times \text{Padding}}{\text{Stride}} + 1$$

입력 크기 5×5,
필터 크기 3×3,
스트라이드 1,
패딩 1



Convolutional Layer



Convolutional 연산은 선형연산이다

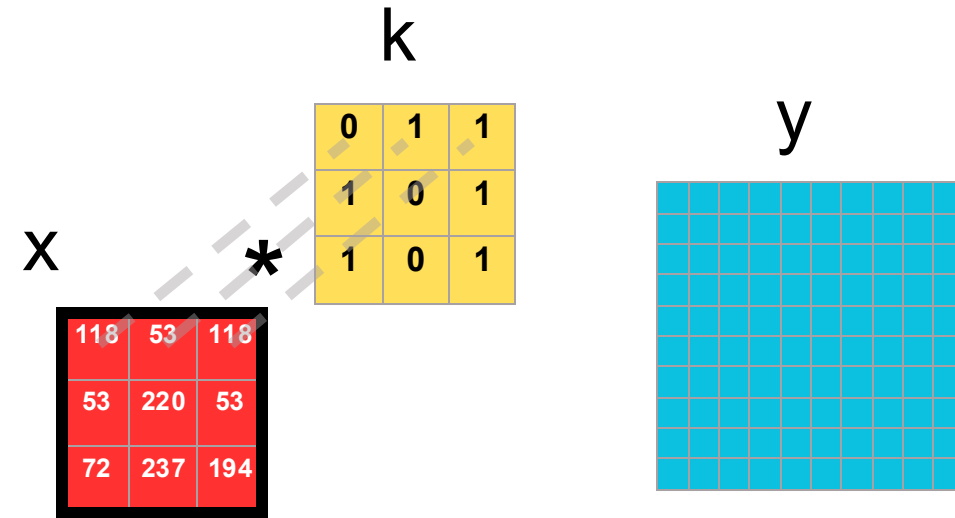
O / X

Convolutional Layer

- 합성곱 연산은 입력 이미지와 필터(커널) 간의 점곱(Summation of Element-Wise Multiplication)으로 계산됩니다.
- 이 과정은 선형적이며, 입력 값과 필터 값의 선형 조합만 포함됩니다

$$y(i, j) = \sum_m \sum_n \sum_c x(i + m, j + n, c) \cdot k(m, n, c) + b$$

- $y(i, j)$: 합성곱 연산 후 위치 (i, j) 에서의 출력 값 (특징 맵의 픽셀 값)
- $x(i + m, j + n, c)$: 입력 데이터(이미지)에서 위치 $(i + m, j + n)$, 채널 c 의 값
- $k(m, n, c)$: 필터(커널)에서 위치 (m, n) , 채널 c 의 가중치
- b : 필터의 편향 값 (bias)
- m, n : 필터(커널)의 가로, 세로 위치
- c : 입력 이미지의 채널 (예: RGB 이미지의 경우 $c = 3$)



$$\begin{aligned} &118 \times 0 + 53 \times 1 + 118 \times 1 + \\ &53 \times 1 + 220 \times 0 + 53 \times 1 + \\ &72 \times 1 + 237 \times 0 + 194 \times 1 \end{aligned} = \boxed{543} \quad y(i, j)$$

Convolutional Layer

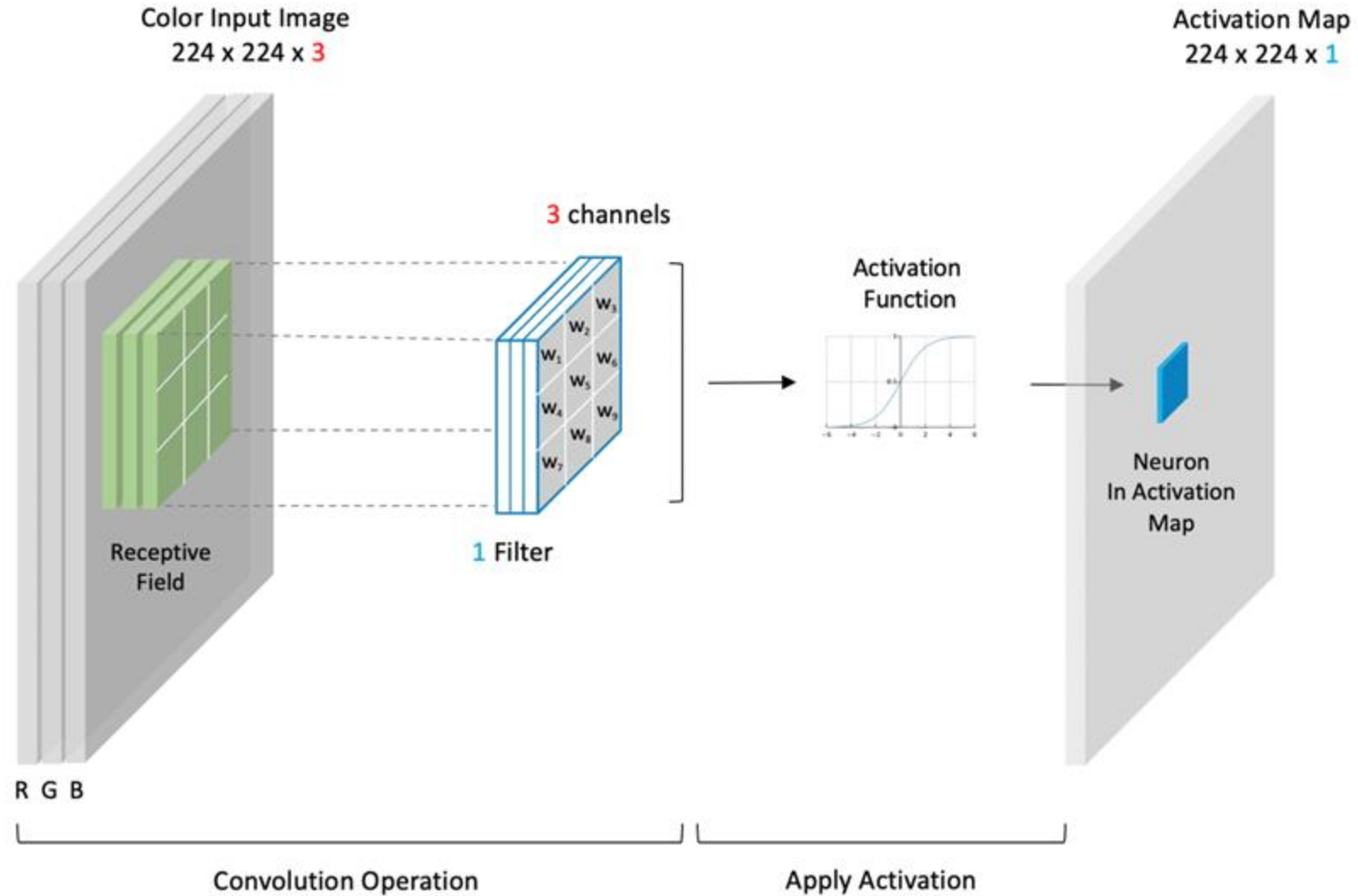
비선형성을 추가하는 과정

- 컨볼루션 연산 후에 **활성화 함수(Activation Function)**를 적용함으로써 비선형성을 추가합니다.
- CNN에서 비선형성을 갖게 되는 핵심 요소는 바로 활성화 함수입니다.
- 활성화 함수(예: ReLU, Sigmoid 등)를 통해 입력 데이터의 선형적 조합에서 벗어나, 복잡한 비선형 관계를 학습할 수 있게 됩니다.

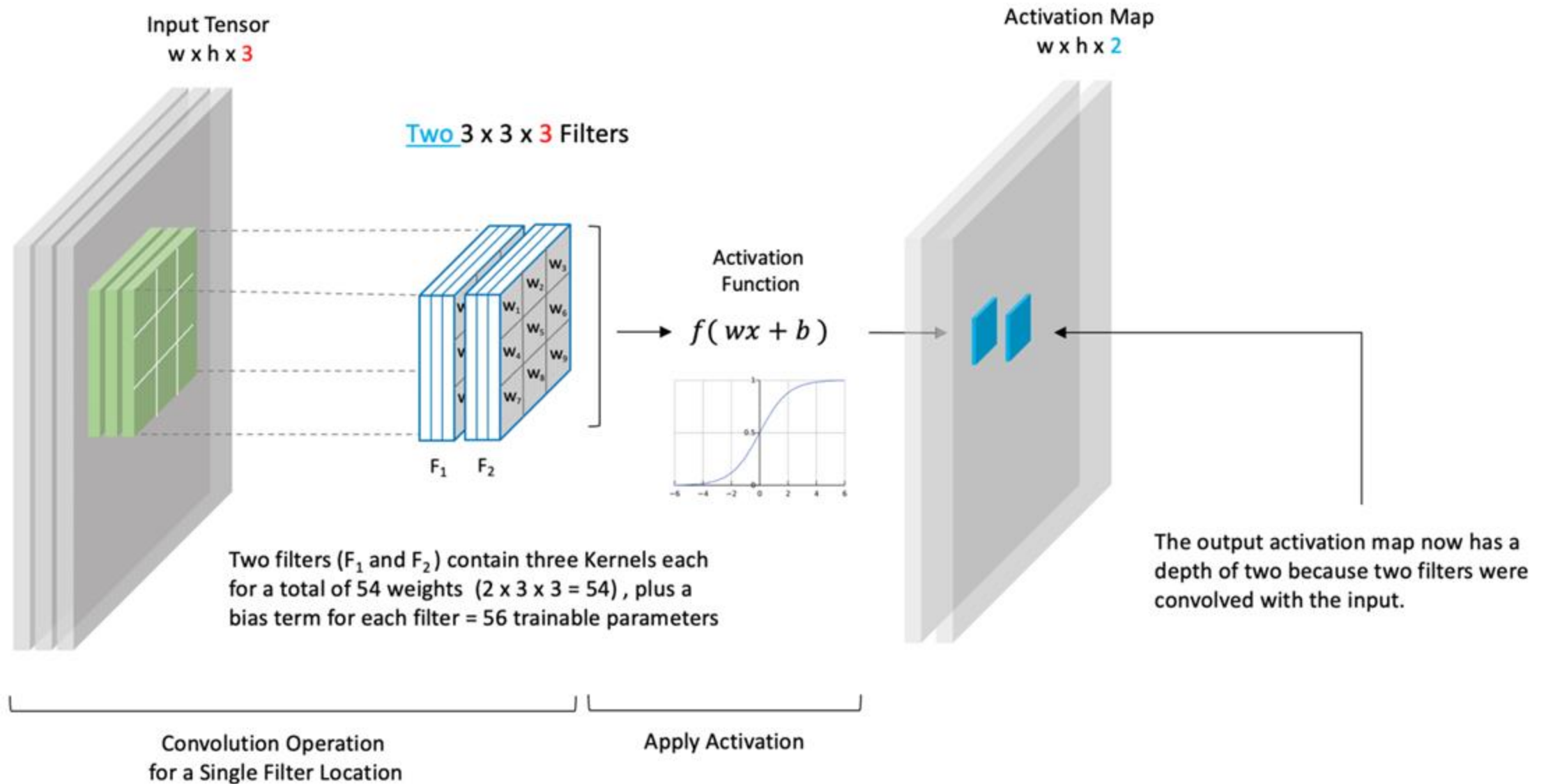
왜 비선형성이 중요한가?

- 비선형성을 추가하지 않으면, 여러 층을 쌓더라도 네트워크 전체는 여전히 선형 변환에 불과합니다.
- 이는 복잡한 데이터 패턴(예: 이미지의 에지, 텍스처 등)을 학습할 수 없게 만듭니다.
- 활성화 함수는 컨볼루션 레이어 이후에 비선형성을 추가하여, 네트워크가 복잡한 데이터 구조를 학습할 수 있도록 합니다.

Convolutional Layer



Convolutional Layer



- 입력 이미지 크기가 128×128×3이고, 5×5 커널이 스트라이드 2, 패딩 2로 적용되었다면, 출력 특징 맵의 크기는?

$$\text{Output Size} = \frac{\text{Input Size} - \text{Filter Size} + 2 \times \text{Padding}}{\text{Stride}} + 1$$

$$(128 - 5 + 2 * 2) / 2 + 1 = (128 - 5 + 4) / 2 + 1 = 127 / 2 + 1$$

$$= 63.5(\text{소수점 아래는 버림}) + 1 = 64 \rightarrow 64 \times 64$$

컨볼루션은 격자 기반(정수 위치) 연산이라, 커널이 끝까지 딱 맞지 않는 마지막 칸은 계산 대상이 아님
공식을 계산할 때 생기는 소수점 이하는 “불완전하게 걸친 위치”를 의미하며, 자연스럽게 **버림(floor)** 처리

Convolution 연산

Convolution Filter를 이용한 연산



Convolutional Layer

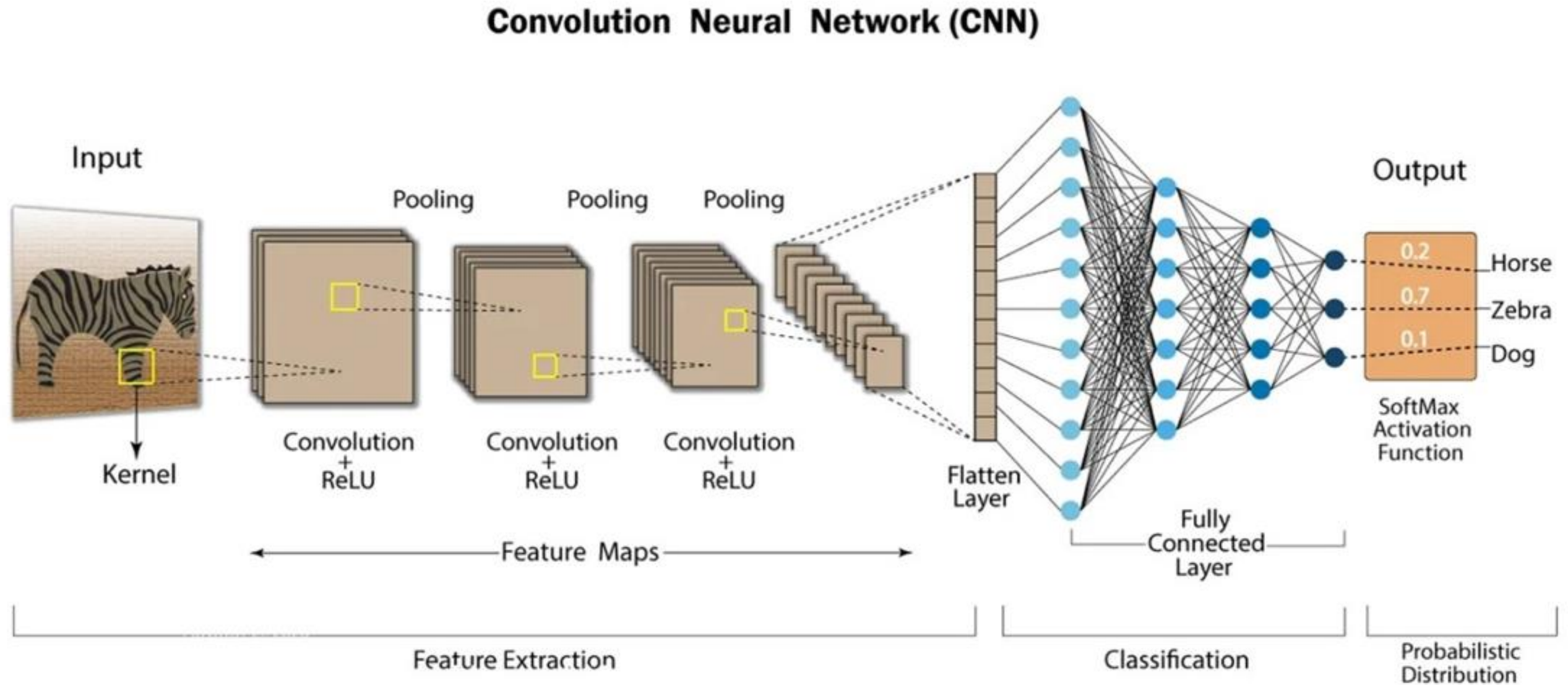


Convolutional Neural Network

Convolutional Neural Network

Convolutional Layer로 이루어진 네트워크 구조

CNN(Convolutional Neural Network)



CNN(Convolutional Neural Network)

CNN의 첫 번째 레이어의 입력 데이터의 크기가
 $64 \times 64 \times 16$ 일 때,

3×3 커널, 32개의 필터를 적용했다면

두 번째 레이어의 입력 차원은?

(stride = 1, valid padding)

$(\text{입력 크기}(64) - \text{필터 크기}(3) + 2 * \text{패딩}(0)) / \text{스트라이드}(1) + 1$

$= (64 - 3 + 0) / 1 + 1 = 61 / 1 + 1 = 61 + 1 = 62$

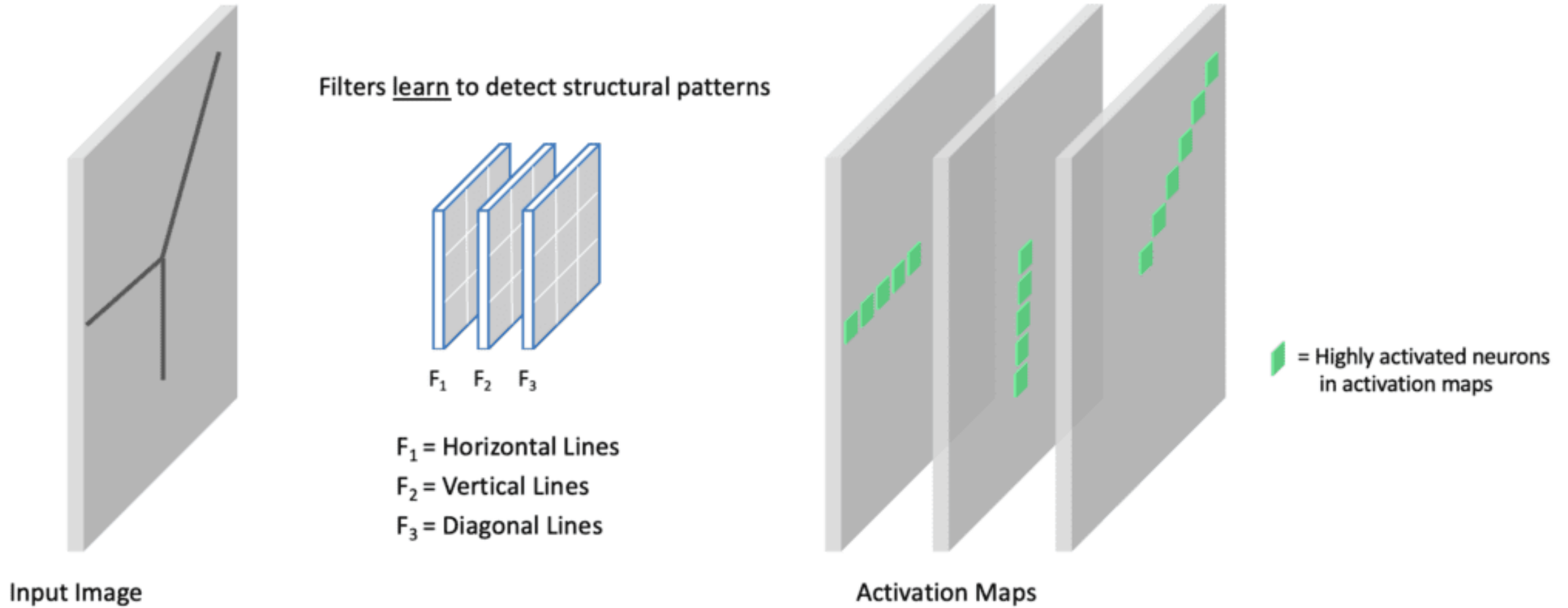
→ **$62 \times 62 \times 32$**

두 번째 CNN 레이어의 입력은?

CNN에서 두 번째 레이어의 입력은 첫 번째 레이어의 출력

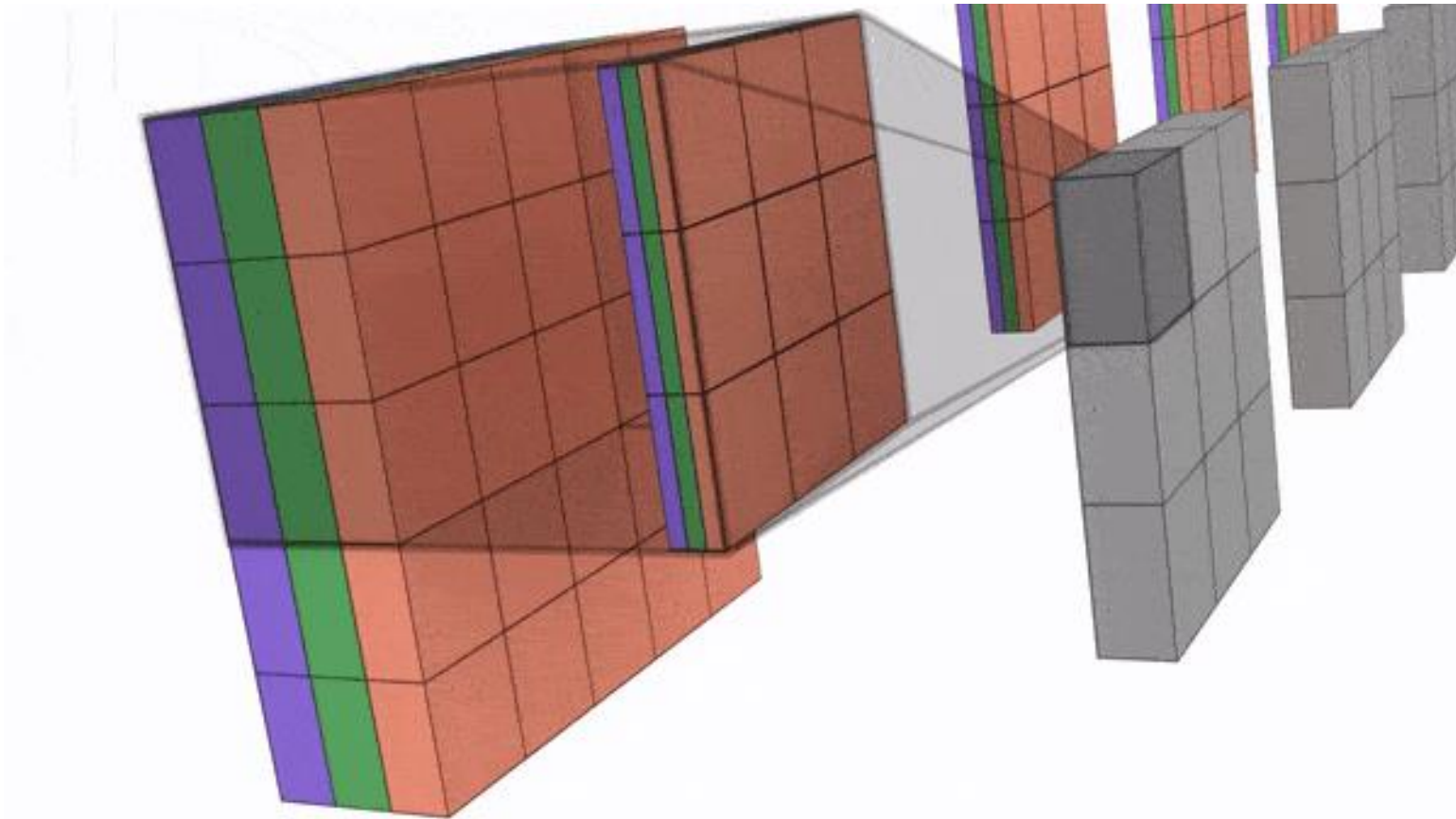
- 특징 맵의 집합:
 - 첫 번째 레이어에서 각 필터가 학습한 패턴이 반영된 특징 맵.
 - 입력 이미지의 국소적 정보가 정리된 데이터.
- 다중 채널 입력:
 - 첫 번째 레이어에서 필터가 여러 개 사용되면 출력 특징 맵도 여러 개 생성됨.
 - 이로 인해 두 번째 레이어의 입력은 다중 채널로 구성. (즉, 이전 층의 커널(필터) 개수 == 입력 채널의 크기)
예: $H \times W \times N$, N = 첫 번째 레이어 필터 수.
- 추상화된 정보:
 - 원본 이미지에서 추출한 에지, 텍스처, 패턴 등 기본적 특징이 포함.

두 번째 CNN 레이어의 입력은?



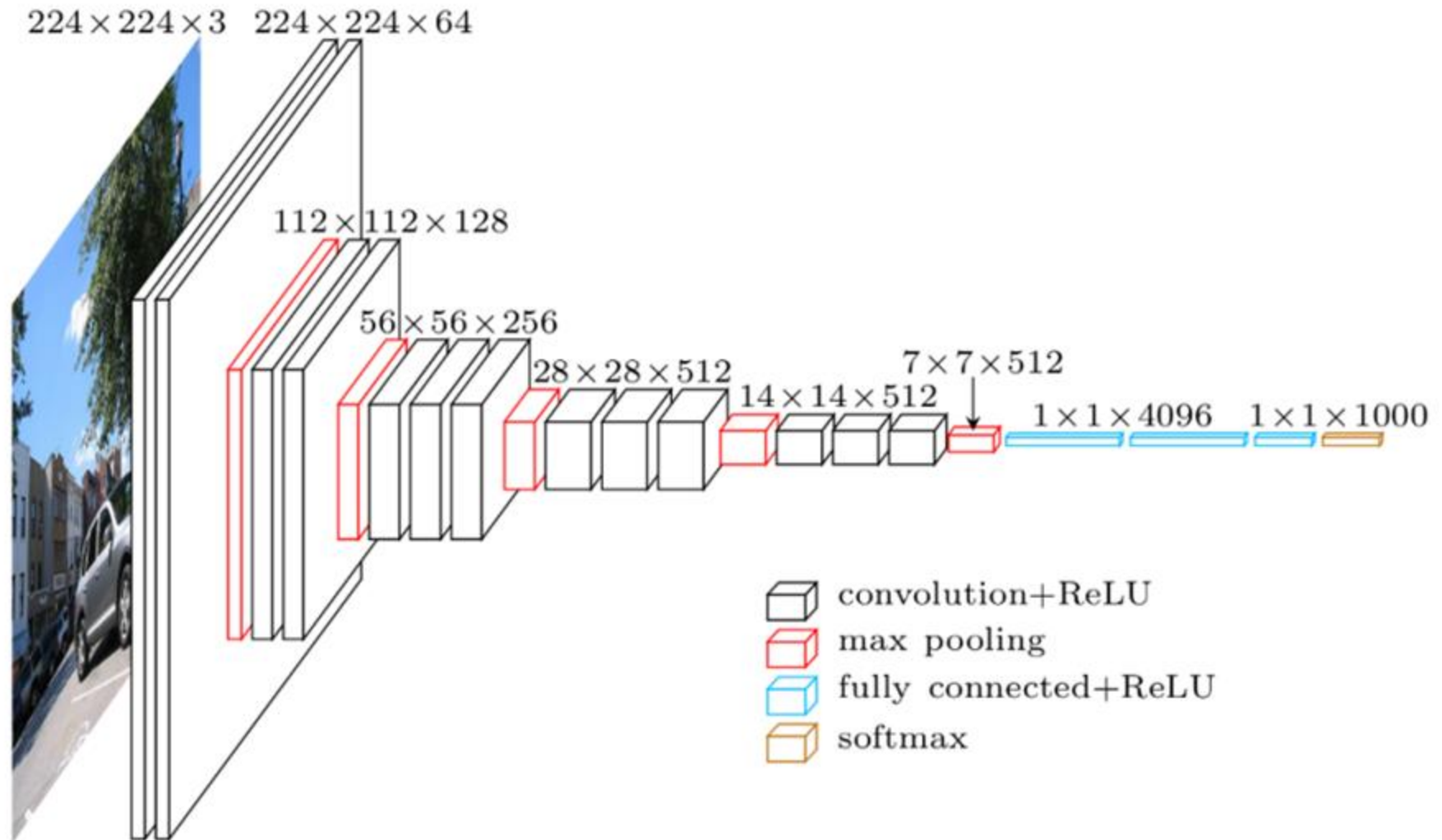
Channel

- Convolutional Layer의 서로 다른 필터들은 각각 적절한 특징(feature) 값을 추출하도록 학습합니다
- 일반적으로 CNN에서 레이어가 깊어질수록 채널의 수가 많아지고 너비와 높이는 줄어듭니다.

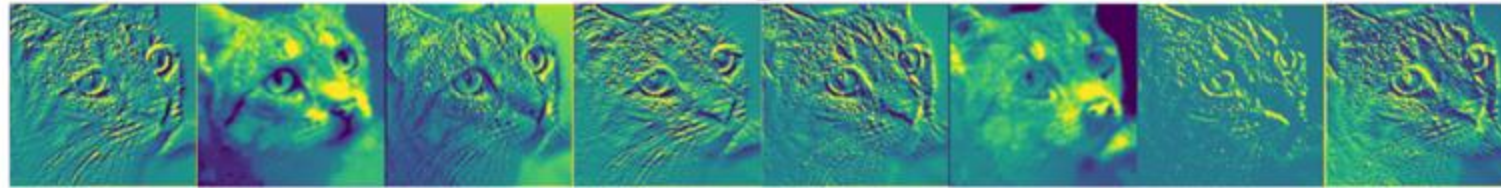


Channel

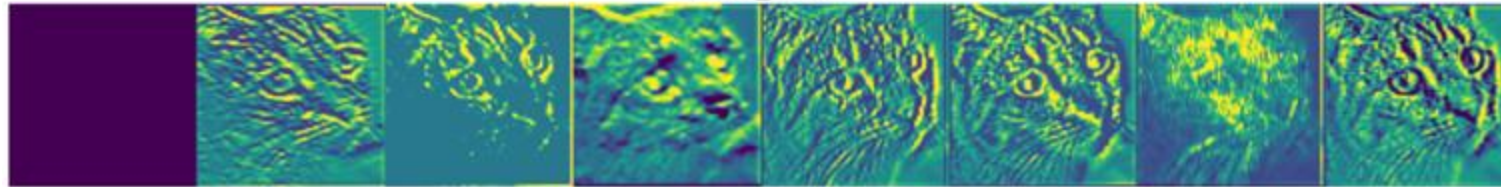
“Very Deep Convolutional Networks for Large-Scale Image Recognition”(Simonyan & Zisserman, 2014) - VGGNet



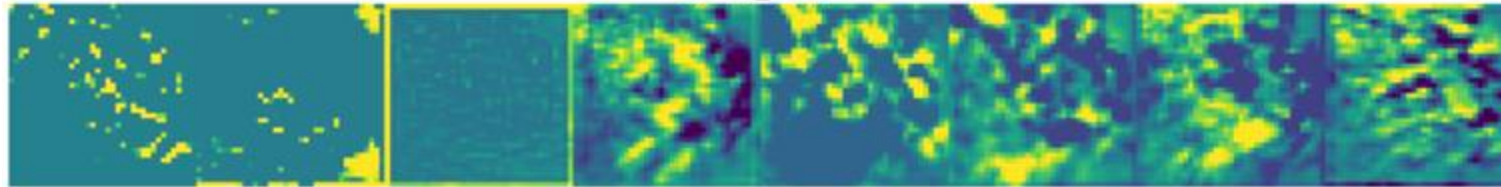
block1_conv1



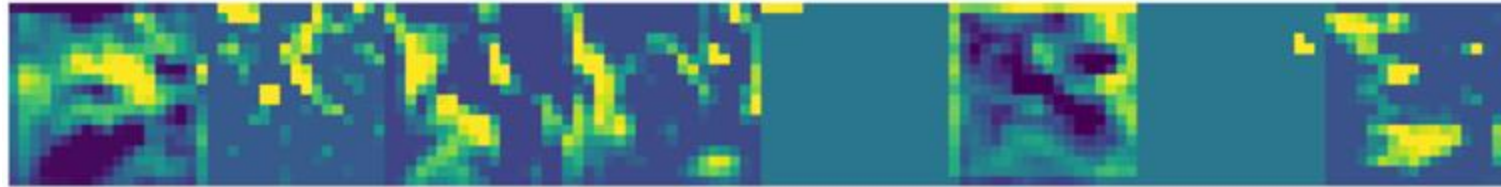
block2_conv1



block3_conv1



block4_conv1



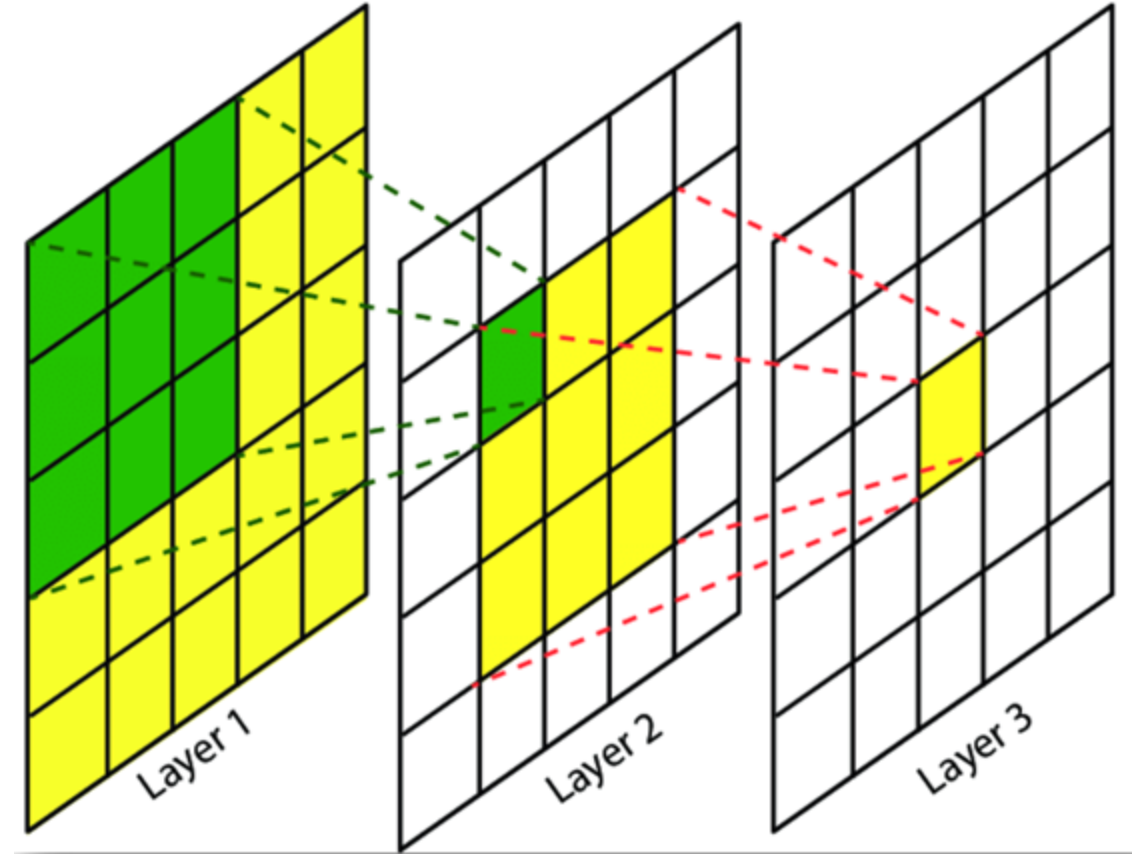
block5_conv1



Receptive Field

Receptive Field

- CNN에서 한 뉴런이 입력 이미지의 어느 범위를 바라보는지(영향받는 영역)를 의미.
- 각 뉴런은 한 픽셀의 정보만 보지 않고, 주변 픽셀의 정보를 함께 고려하여 특징을 학습.
- 입력 이미지의 일부 영역을 학습하며, 깊은 네트워크를 통해 전역적인 특징까지 확장됨.
- Receptive Field가 크면 모델이 더 넓은 정보(문맥)을 활용 가능!



Receptive Field

- CNN의 각 층에서 필터(커널)가 적용되면서 **Receptive Field**가 점진적으로 확대됨.
- 초기 층: 작은 패턴(에지, 선 등) 인식.
- 깊은 층: 넓은 영역의 정보를 반영하여 복잡한 패턴(물체의 모양, 질감 등) 학습

$$RF_l = RF_{l-1} + (F_l - 1) \times \prod_{j=1}^{l-1} S_j$$

- RF_l : 현재 층에서의 Receptive Field 크기
- RF_{l-1} : 이전 층의 Receptive Field 크기
- F_l : 현재 층의 필터(커널) 크기
- S_{l-1} : 이전 층의 스트라이드 값

Receptive Field

- Stride가 클수록 RF 증가 속도도 빨라짐

$$RF_l = RF_{l-1} + (F_l - 1) \times \prod_{j=1}^{l-1} S_j$$

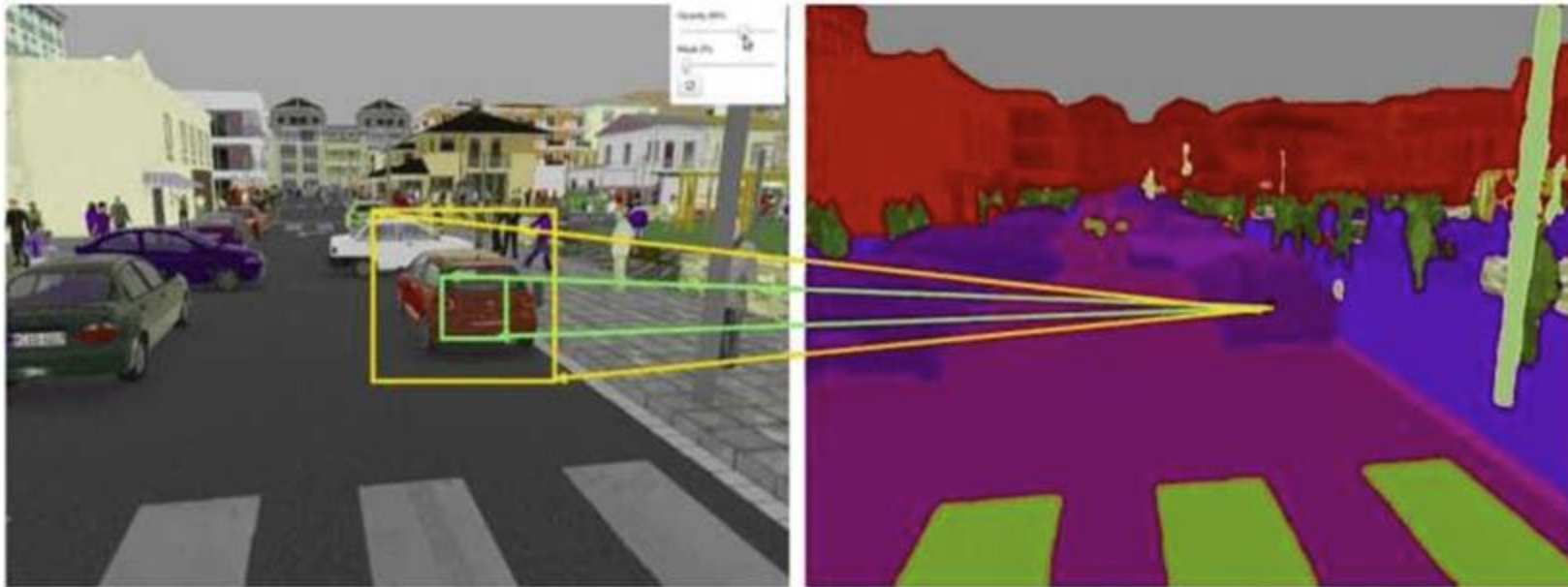
- RF_l : 현재 층에서의 Receptive Field 크기
- RF_{l-1} : 이전 층의 Receptive Field 크기
- F_l : 현재 층의 필터(커널) 크기
- S_{l-1} : 이전 층의 스트라이드 값

층	필터(F)	스트라이드	누적 스트라이드	입력 RF 크기	출력 RF 크기
Conv1	3	1	1	1	$1+(3-1) \times 1 = 3$
Conv2	3	2	1	3	$3+(3-1) \times 1 = 5$
Conv3	3	2	2 (1x2)	5	$5+(3-1) \times 2 = 9$
Conv4	3	2	4 (1x2x2)	9	$9+(3-1) \times 4 = 17$

Receptive Field

Receptive Field

- CNN의 각 층에서 필터(커널)가 적용되면서 **Receptive Field**가 점진적으로 확대됨.
- 초기 층: 작은 패턴(에지, 선 등) 인식.
- 깊은 층: 넓은 영역의 정보를 반영하여 복잡한 패턴(물체의 모양, 질감 등) 학습.



Pooling Layer

다운샘플링을 위한 연산을 수행하는 레이어



Pooling Layer

Pooling Layer

- 특징 맵(Feature Map)의 차원을 축소하여 계산량을 감소
- 공간적 불변성을 증가시켜 작은 변화에도 강한 특징을 학습
- 서브 샘플링(Sub Sampling)이라고도 하며, 일반적으로 2×2 또는 3×3 필터 사용
- 풀링 연산도 컨볼루션 연산처럼 커널을 이동하며 진행하지만, 가중치 없이 사이즈와 종류만 존재
- 보통 풀링 크기와 스트라이드 크기를 동일하게 설정하여 차원 축소 효과 극대화

Max Pooling

Take the **highest** value from the area covered by the kernel

Average Pooling

Calculate the **average** value from the area covered by the kernel

Example: Kernel of size 2×2 ; stride=(2,2)

3	2	0	0
0	7	1	3
5	2	3	0
0	9	2	3

Convolved Feature (4 x 4)

	7		

Output

Max values

3	2	0	0
0	7	1	3
5	2	3	0
0	9	2	3

Convolved Feature (4 x 4)

	3		

Output

Average values

Pooling Layer

특징	맥스 풀링 (Max Pooling)	에버리지 풀링 (Average Pooling)
정보 선택	윈도우 내 가장 큰 값 선택	윈도우 내 모든 값의 평균 계산 (요약)
물리적 의미	가장 두드러진 특징 강조 → 날카로운 엣지·패턴 보존	영역 전체를 부드럽게 요약 → 전반적 통계치 유지
정보 보존 특성	강한 특징 정보는 유지, 약한 신호는 사라질 수 있음	모든 특징이 부분적으로 반영되지만, 값이 희석(blur)될 가능성
민감도	특징의 정확한 위치보다 존재 유무에 민감	평균이므로 극단값에 둔감 ⇒ 출력이 매끄럽고 노이즈 저항↑
주요 사용처	일반적인 CNN 특징 추출 (분류·객체 탐지)	글로벌 Average Pooling(GAP)·과적합 완화·FC 대체 등



Pooling Layer

공간적 불변성(Positional Invariance)

이미지 내의 특정한 특징이 약간 이동하거나 위치가 변해도 그 특징을 인식할 수 있는 능력

A, B, C 이미지가 2x2 커널, 스트라이드 2인 MaxPooling 층을 통과한다고 했을 때,

B, C는 A를 오른쪽으로 픽셀을 1, 2픽셀 이동

이 때, A와 B의 출력값은 동일하고, C도 50% 동일

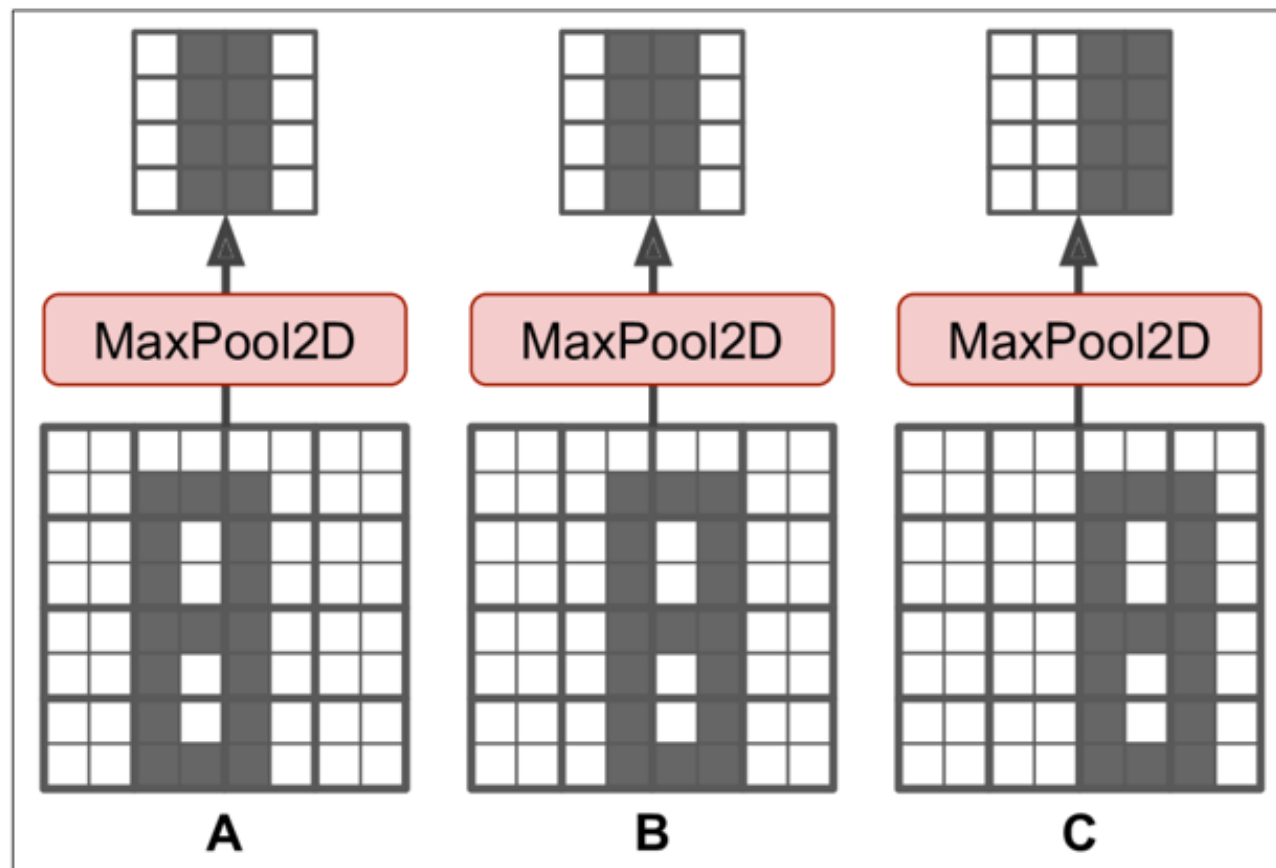


Figure 14-9. Invariance to small translations

Pooling Layer의 장단점

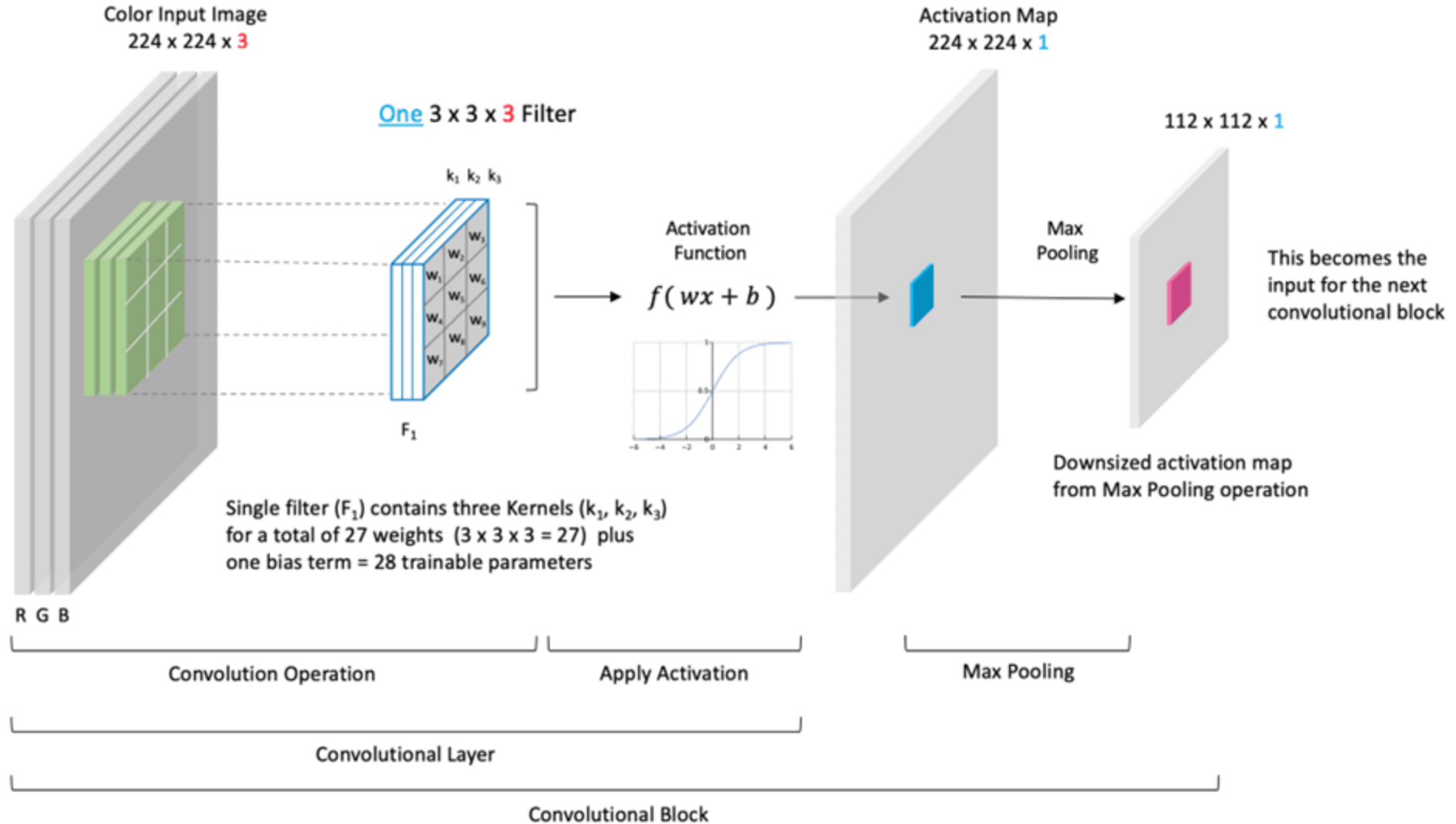
✅ 장점:

- 계산량 감소 → 모델 학습 속도 향상
- 과적합 방지 → 더 일반화된 특징 학습
- 위치 불변성 강화 → 작은 변형에도 강한 특징 유지

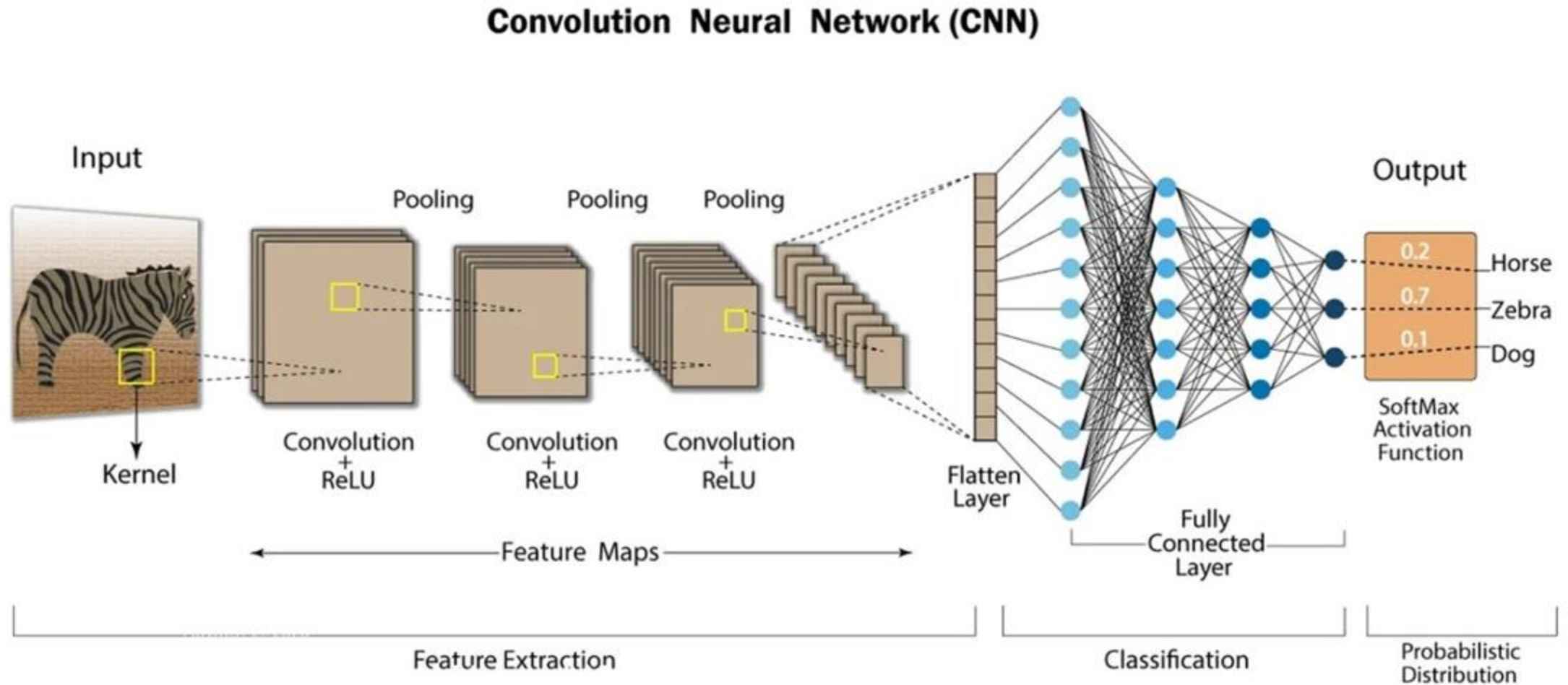
❌ 단점:

- 정보 손실 가능성 → 중요한 세부 정보가 사라질 수 있음
- 객체의 위치 정보 손실 → 공간적 정밀도가 낮아질 가능성
- 고정된 연산 방식 → 모든 데이터에 최적화되지 않을 수 있음

Pooling Layer

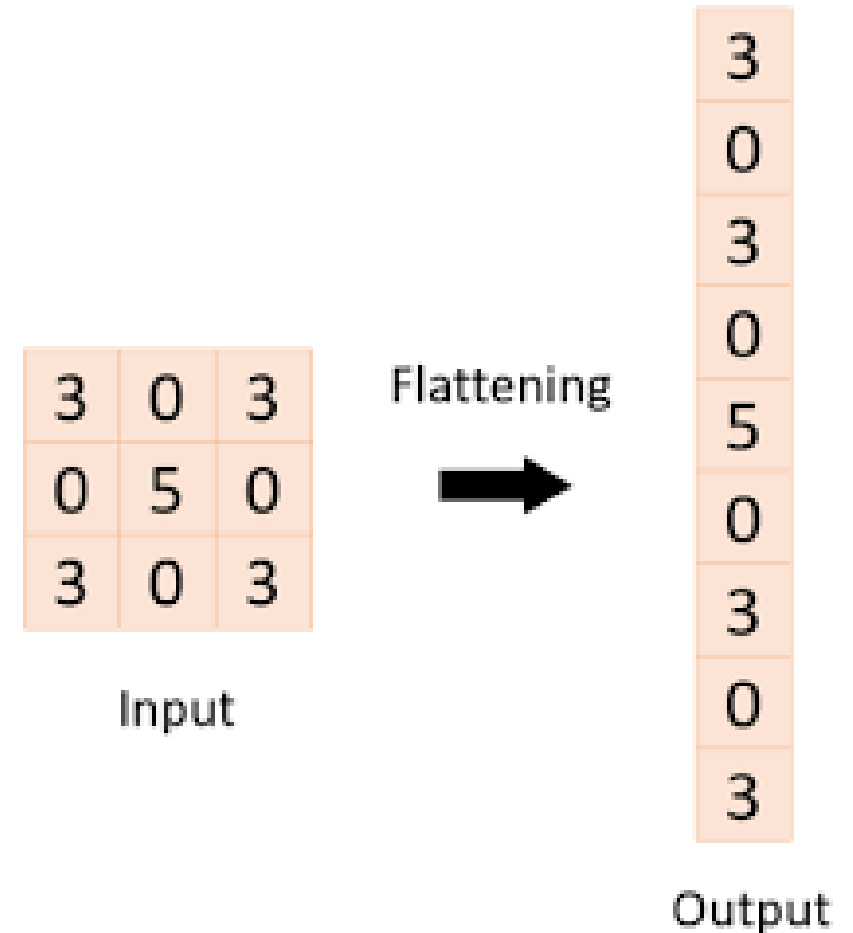


CNN(Convolutional Neural Network)



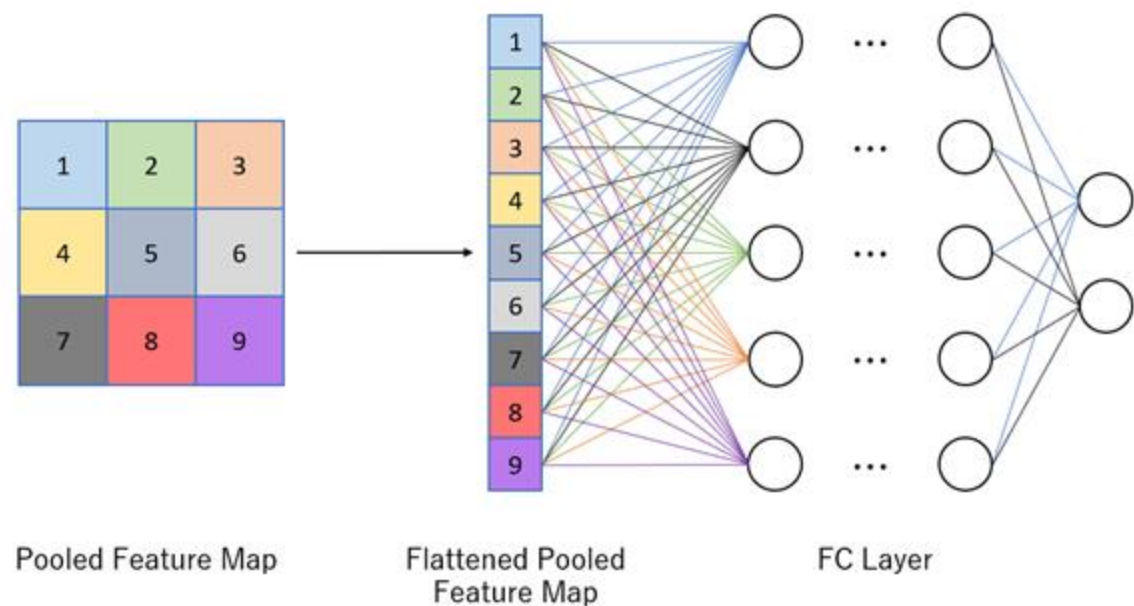
Flatten Layer

- 특징 맵(Feature map, Activation map)을 1차원 벡터로 변환하는 레이어
- CNN(합성곱 신경망)에서 주로 사용되며, 완전 연결 레이어(Fully Connected Layer)와의 연결을 위해 필요
- 입력 특징 맵 (3D) → Flatten Layer → Fully Connected Layer



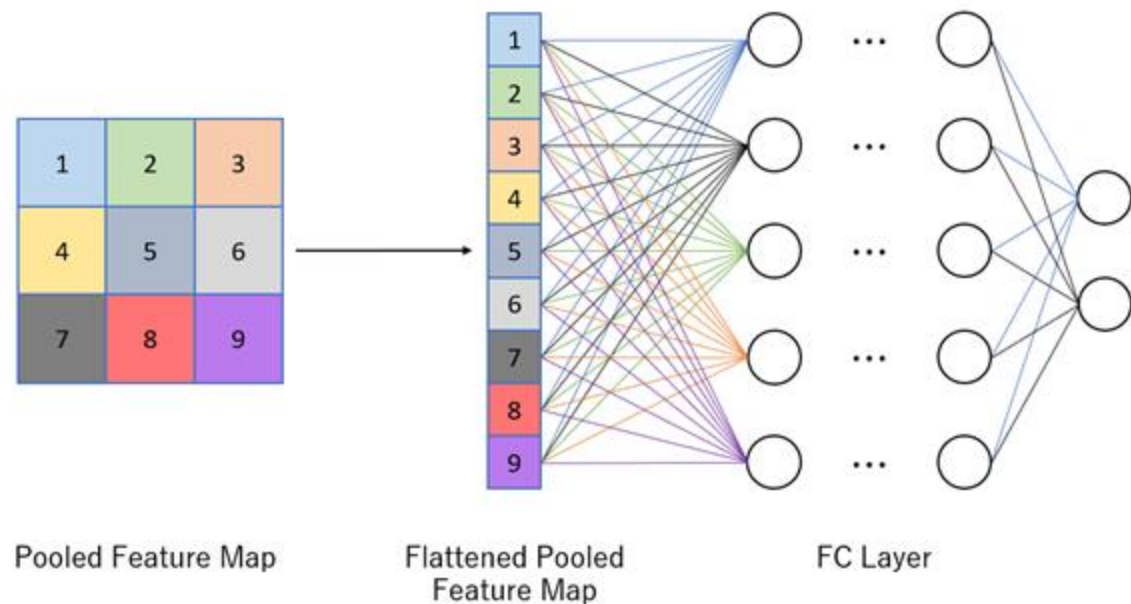
Fully Connected Layer

- 이전 레이어의 특징들을 조합하여 고수준 특징을 학습하는 레이어
- 컨볼루션 레이어에서 추출된 공간적 특징을 전역적으로 결합
- 최종 분류나 회귀를 위한 출력을 생성
- 다양한 입력-출력 매핑을 학습할 수 있음
- 일반적으로 CNN의 마지막 부분에 위치 (컨볼루션 레이어와 풀링 레이어 이후)

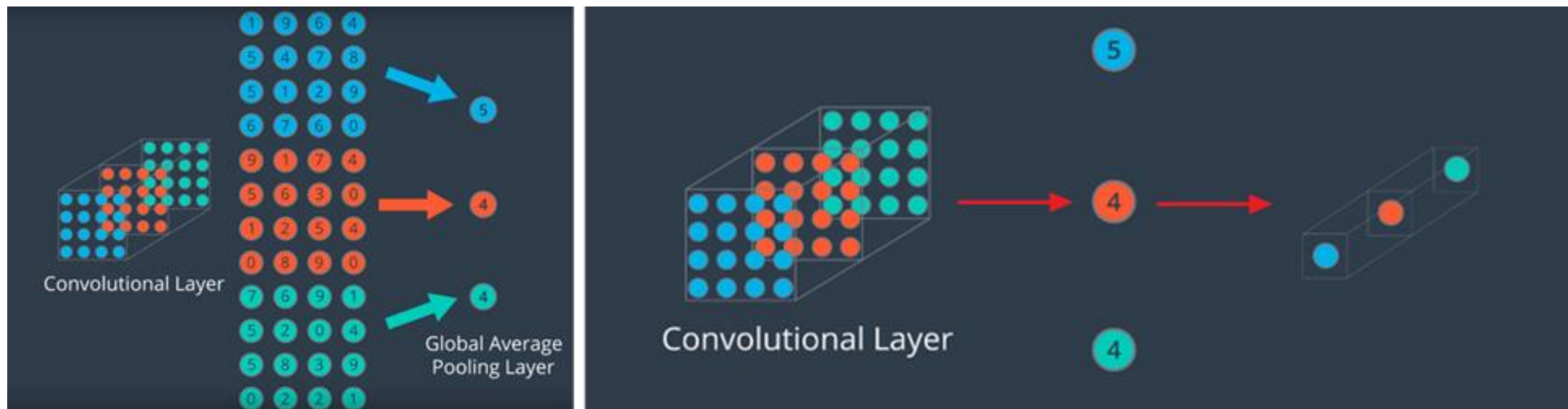


Fully Connected Layer

- 과적합의 위험: 드롭아웃(Dropout) 등의 정규화 기법 사용
- 입력의 공간적 구조를 무시: 컨볼루션과 다르게 공간적 관계가 고려되지 않음
- 대체 기법: 일부 모델에서는 FC 레이어 대신 Global Average Pooling(GAP) 사용

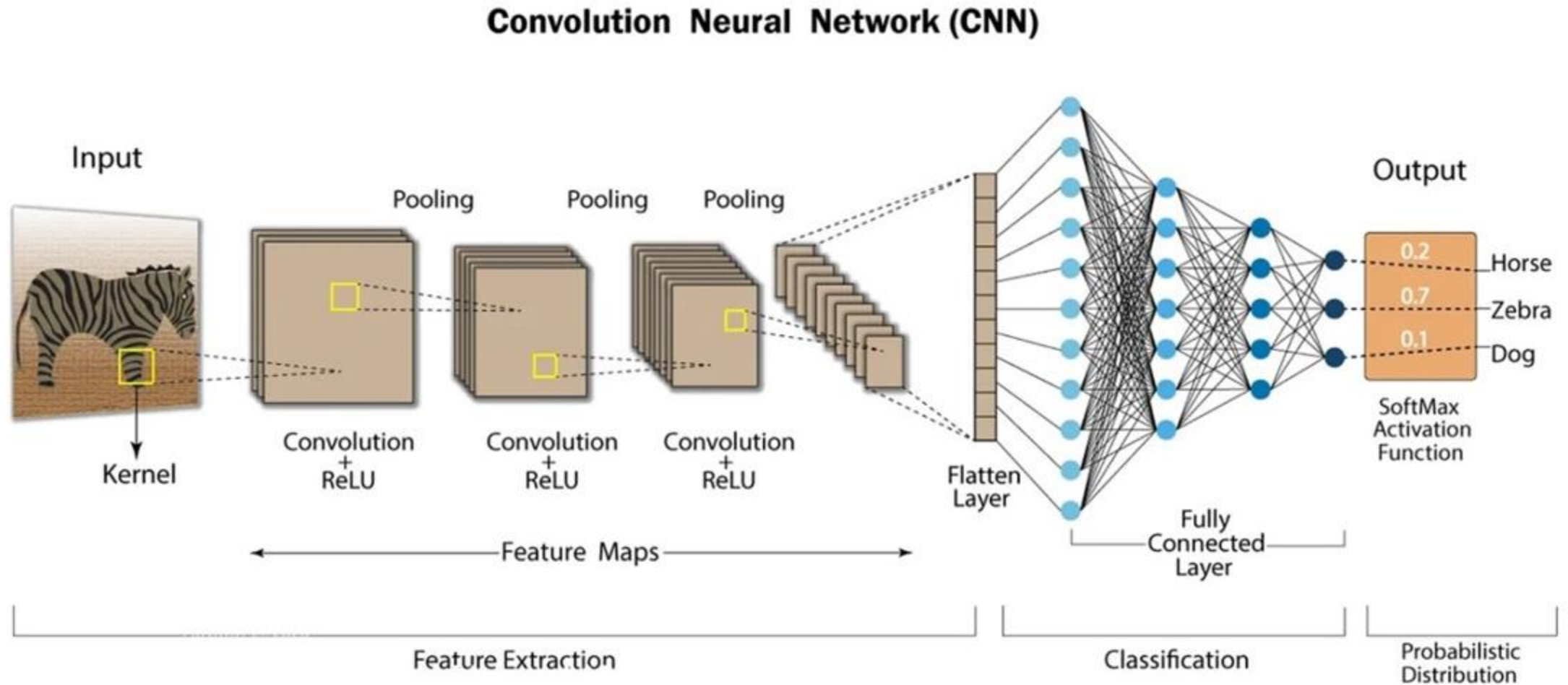


Global Average Pooling (GAP)



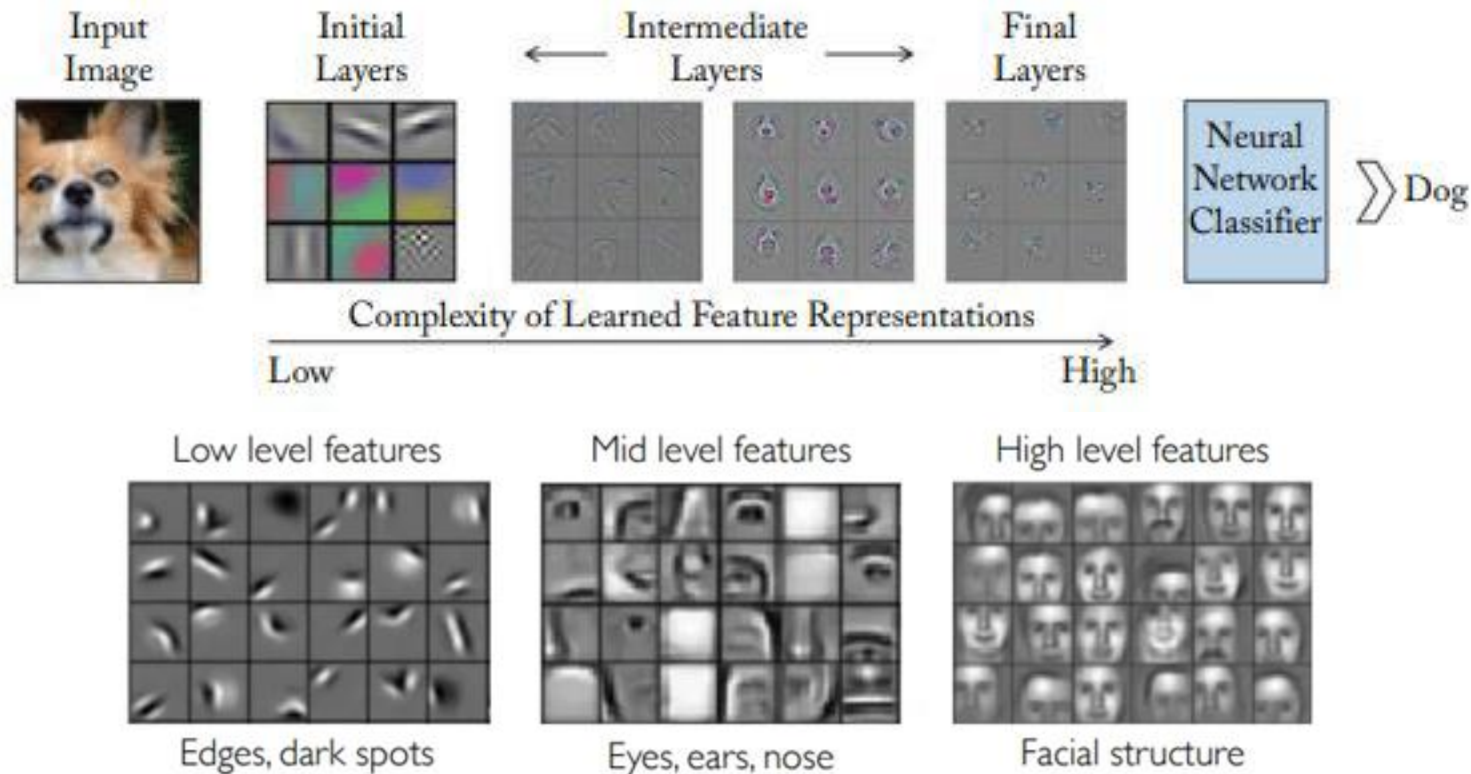
- GAP는 CNN + FC(Fully Connected) Layer에서 classifier인 FC Layer를 없애기 위한 방법으로 도입
- FC Layer는 마지막 feature와 matrix 곱을 하여 feature 전체를 연산의 대상으로 삼아서 결과를 출력
 - a. FC layer를 classifier로 사용하는 경우 파라미터의 수가 많이 증가
 - b. feature 전체를 matrix 연산하기 때문에 위치에 대한 정보도 사라지짐
- 단순히 $(H, W, C) \rightarrow (1, 1, C)$ 크기로 줄어드는 연산이므로 파라미터가 추가되지 않음
- 파라미터의 갯수가 FC Layer 만큼 폭발적으로 증가하지 않아서 over fitting 측면에서도 유리.
- GAP 연산 결과 1차원 벡터가 되기 때문에 최종 출력에 FC Layer 대신 사용

CNN(Convolutional Neural Network)



CNN(Convolutional Neural Network)

- Convolutional Neural Network(CNN)은 시각 피질의 작동 방식에서 영감을 받은 인공 신경망
- 이미지를 처리할 때 특징을 추출하는 작은 필터를 이용하여 이미지 전체를 스캔



대표적 CNN 모델

LeNet

Lecun, Y.; Bottou, L.; Bengio, Y.; Haffner, P. (1998).

"Gradient-based learning applied to **document recognition**"

미국 우편 공사의 우편번호 손글씨 인식



MNIST database

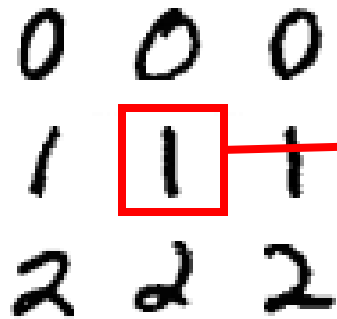
미국 우편 공사의 우편번호 손글씨 인식



MNIST database

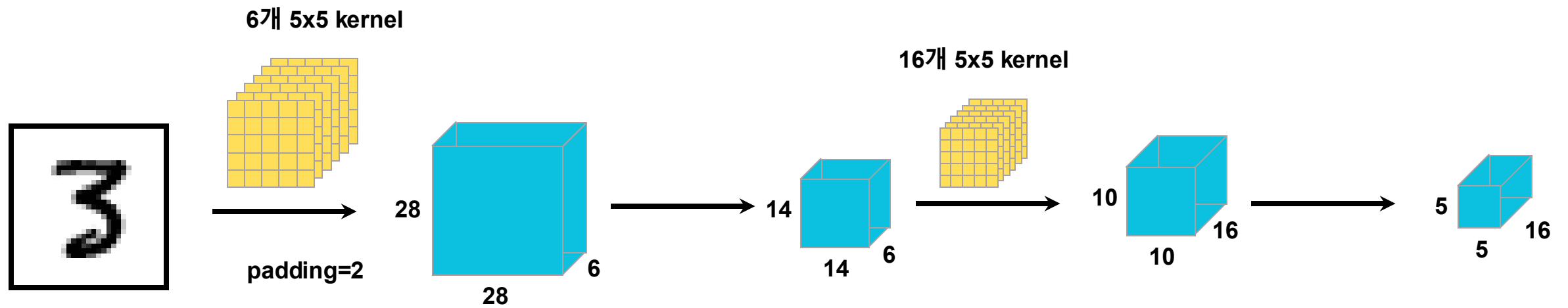


MNIST database

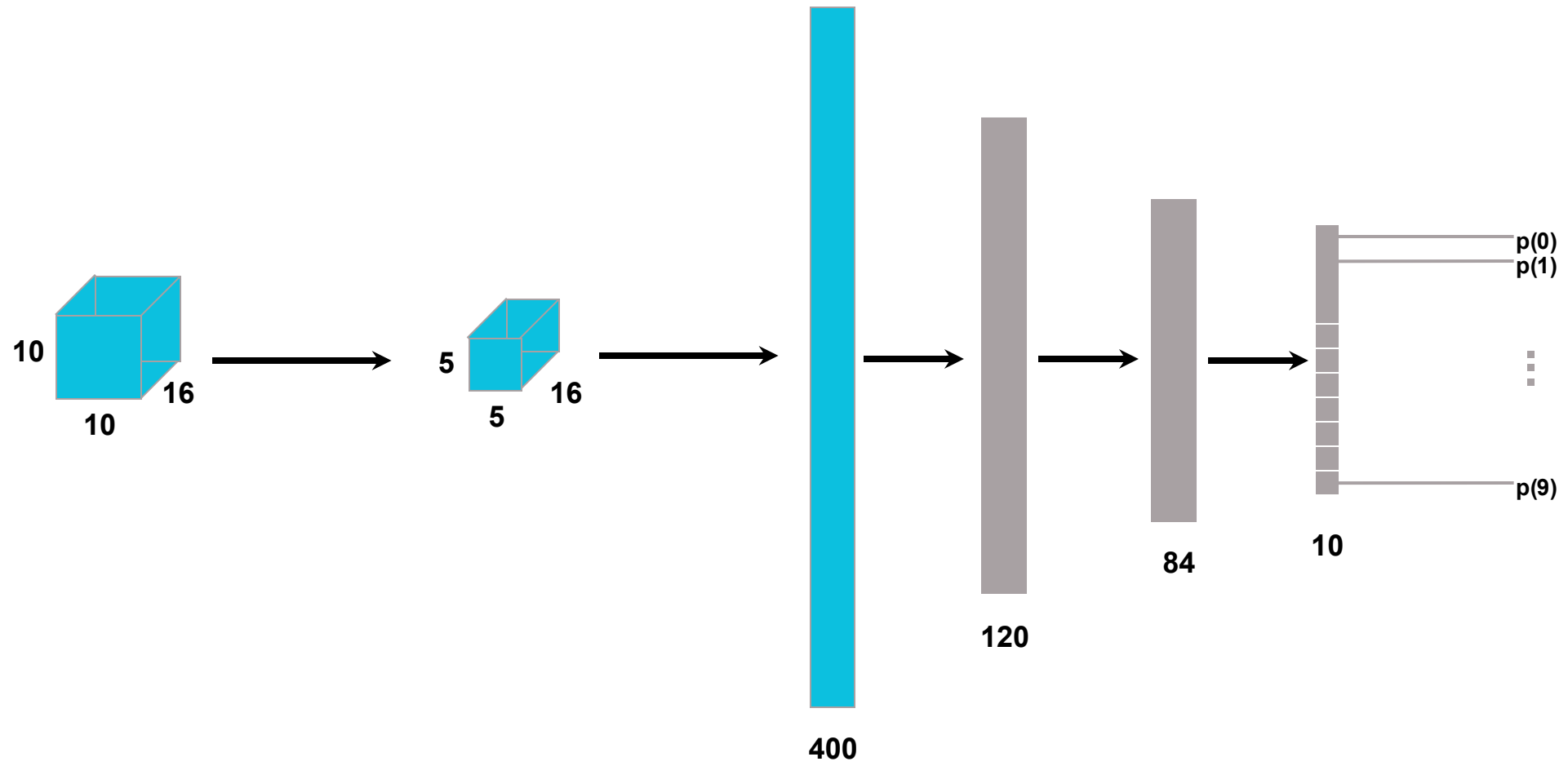


- 28 x 28 픽셀
- 흑백톤 (Grayscale)
- 학습 데이터 60,000 /
테스트 데이터 10,000

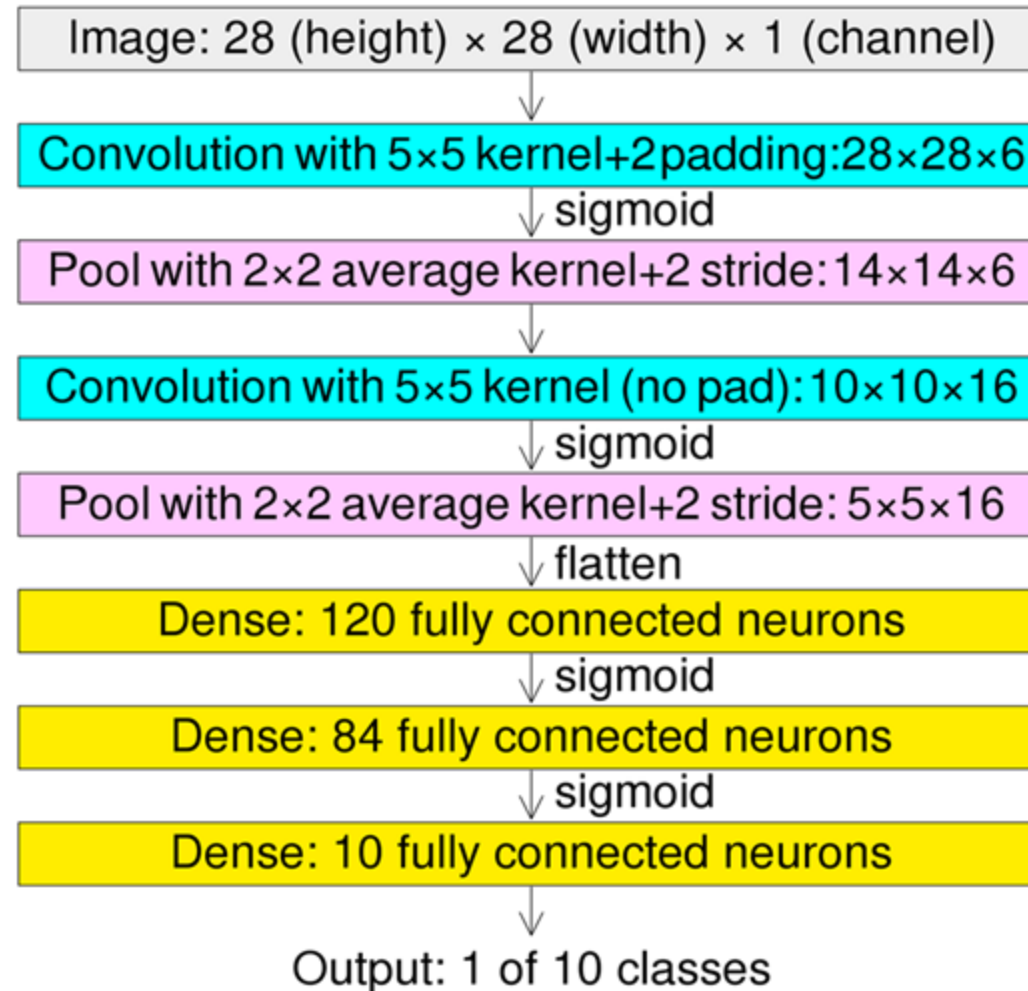
LeNet



LeNet



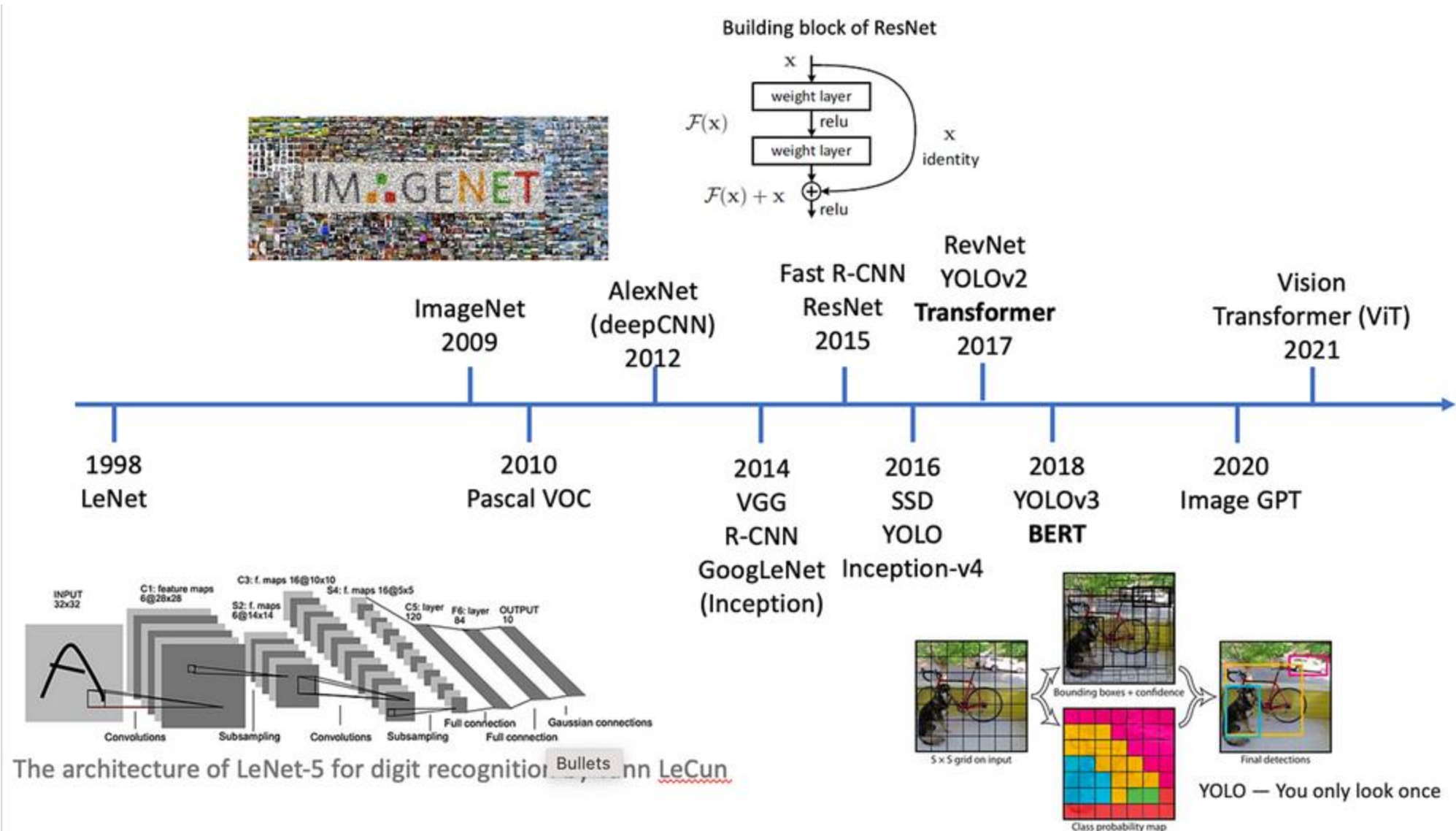
LeNet



LeNet

- 딥 러닝 > 다른 머신 러닝
- 이미지 인식에 효과적 (98% 정확도)
- 딥러닝 여러 연구에서 벤치마크로 활용되는 데이터셋
제시

A brief history of deep learning models in computer vision



<https://youtu.be/jDe5BAsT2-Y>

<https://youtu.be/z9hJzduHToc>